

计算智能与启发式算法

硕士生课程讲义

第五部分：群体智能计算

第一节：群体智能

群体智能

- ❧ 群体智能这个概念来自对蚂蚁、鸟群、蜜蜂的观察。一组相互之间可以进行直接通信或者间接通信(通过改变局部环境)的主体，这组主体能够合作进行分布问题求解。
- ❧ 任何启发于群居性昆虫群体和其它动物群体的集体行为而设计的算法和分布式问题解决装置都称为群体智能。
- ❧ 群体智能在没有集中控制并且不提供全局模型的前提下，为寻找复杂的分布式问题的解决方案提供了基础。

群体智能

- ✧ Millonas在开发人工生命算法时(1994年)，提出群体智能概念并提出五点原则：
- ✧ ①接近性原则：群体应能够实现简单的时空计算；
- ✧ ②优质性原则：群体能够响应环境要素；
- ✧ ③变化相应原则：群体不应把自己的活动限制在一狭小范围；
- ✧ ④稳定性原则：群体不应每次随环境改变自己的模式；
- ✧ ⑤适应性原则：群体的模式应在计算代价值得的时候改变。

群体智能特点

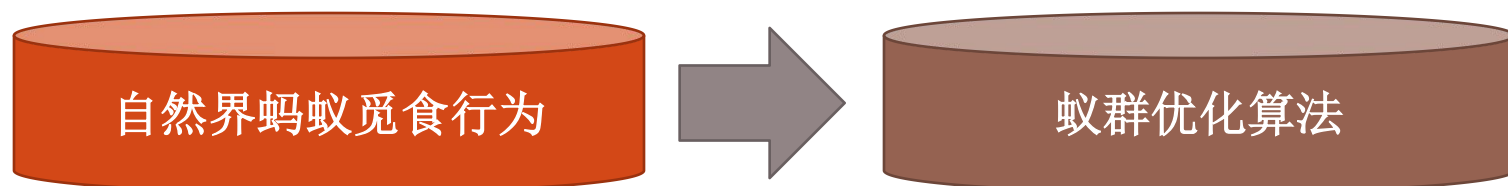
- ❧ 分布式：能够适应当前网络环境下的工作状态；
- ❧ 鲁棒性：没有中心的控制与数据，个体的故障不影响整个问题的求解；
- ❧ 扩充性：个体的增加，系统的通信开销增加小；
- ❧ 简单性：个体简单，实现也比较简单。

群体智能主要算法

- ❧ 蚁群算法 (Ant Colony Optimization, ACO) 是对蚂蚁群落食物采集过程的模拟，已成功应用于许多离散优化问题。
- ❧ 粒子群算法 (Particle Swarm Optimization, PSO)：起源于对简单社会系统的模拟，最初是模拟鸟群觅食的过程，但后来发现它是一种很好的优化工具。

第二节：蚁群算法

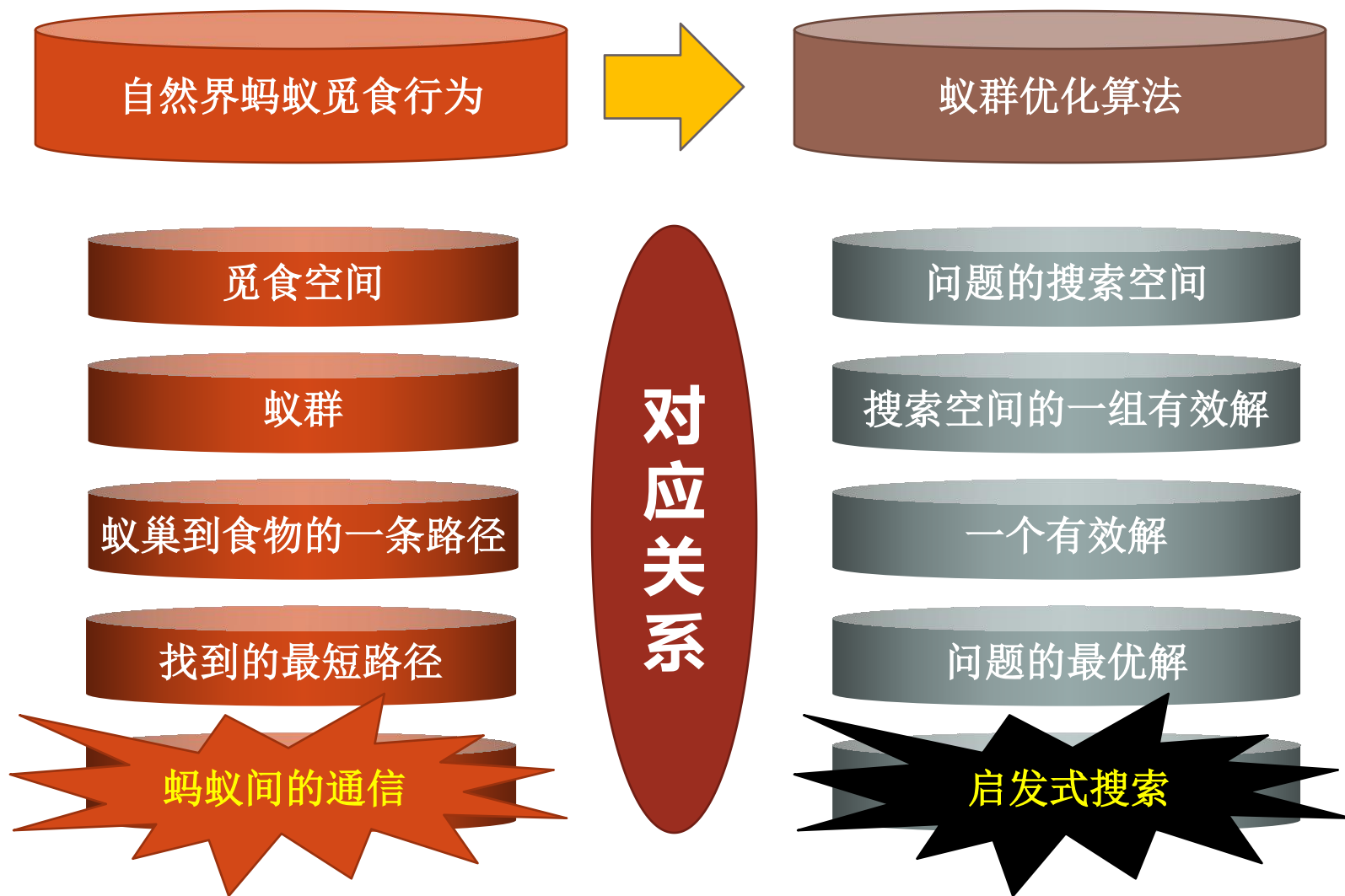
蚁群算法概述



- ❧ 20世纪90年代意大利学者M. Dorigo, V. Maniezzo, A. Coloni等从生物进化的机制中受到启发提出一种新型的模拟进化算法
- ❧ 自然界蚂蚁群体在寻找食物的过程中，通过一种被称为信息素（Pheromone）的物质实现相互的间接通信，从而能够合作发现从蚁穴到食物源的最短路径。
- ❧ 通过对这种群体智能行为的抽象建模，研究者提出了蚁群优化算法（Ant Colony Optimization, ACO），为最优化问题、尤其是组合优化问题的求解提供了一强有力的手段。

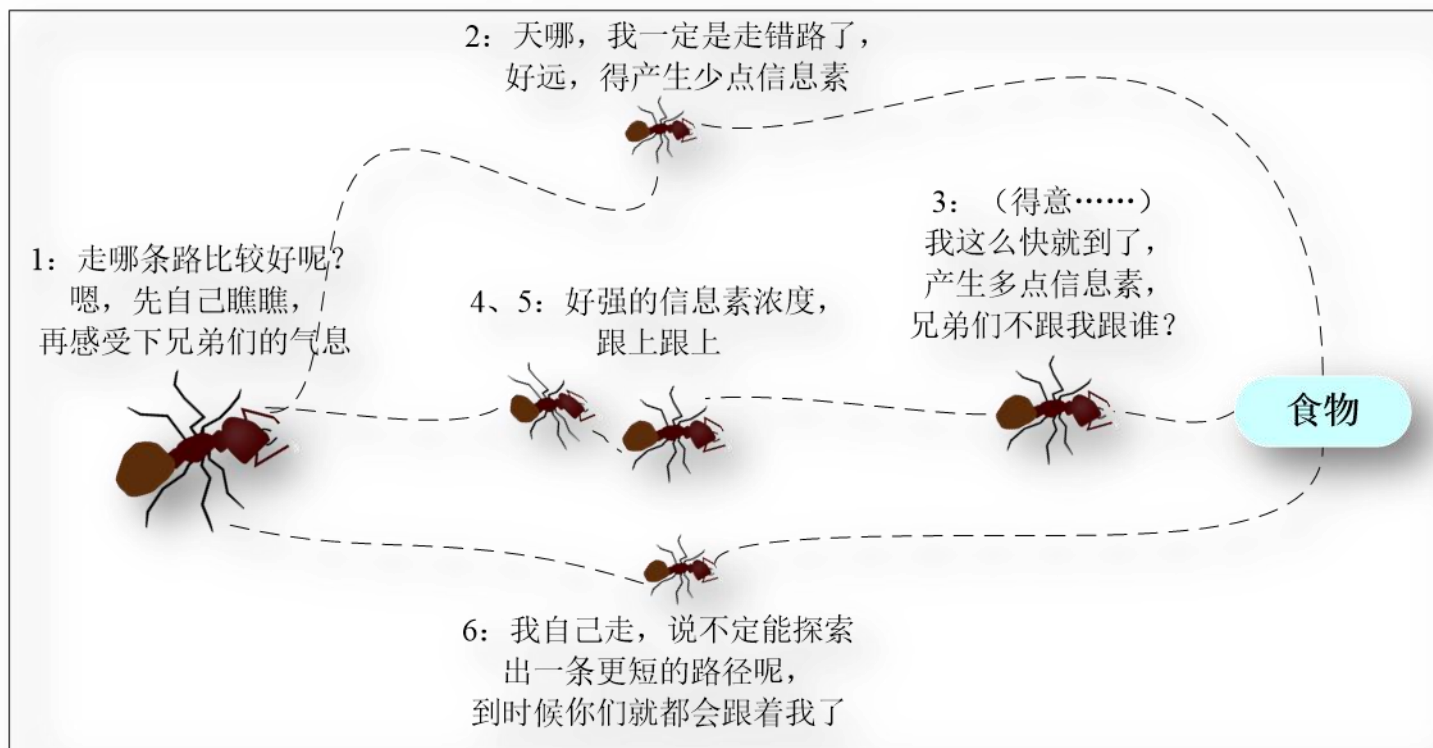


基本原理

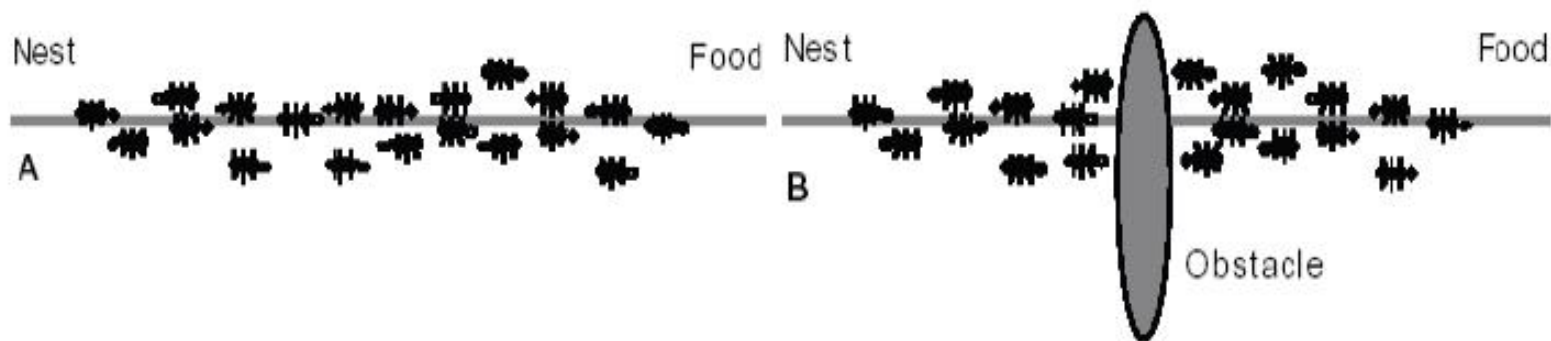


蚂蚁觅食原理

- ❧ 蚂蚁在运动过程中，能够在它所经过的路径上留下一种称之为外激素(pheromone)的物质进行信息传递，而且蚂蚁在运动过程中能够感知这种物质，并以此指导自己的运动方向。
- ❧ 因此由大量蚂蚁组成的蚁群集体行为便表现出一种信息正反馈现象：某一路径上走过的蚂蚁越多，则后来者选择该路径的概率就越大。



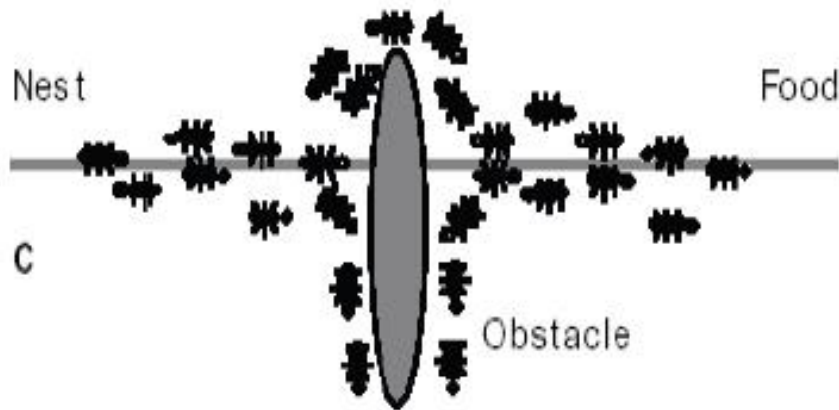
蚂蚁觅食原理



A、无障碍时，蚂蚁到达目标。

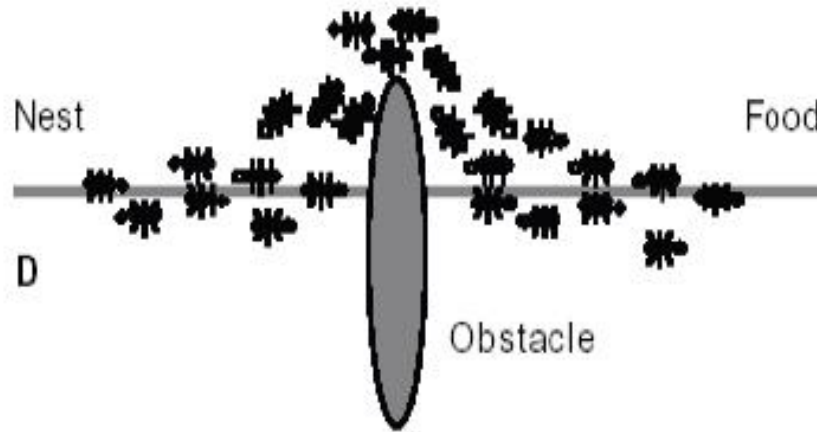
B、一些蚂蚁选择上方路径，一些蚂蚁选择下方路径。选择是随机的。

蚂蚁觅食原理



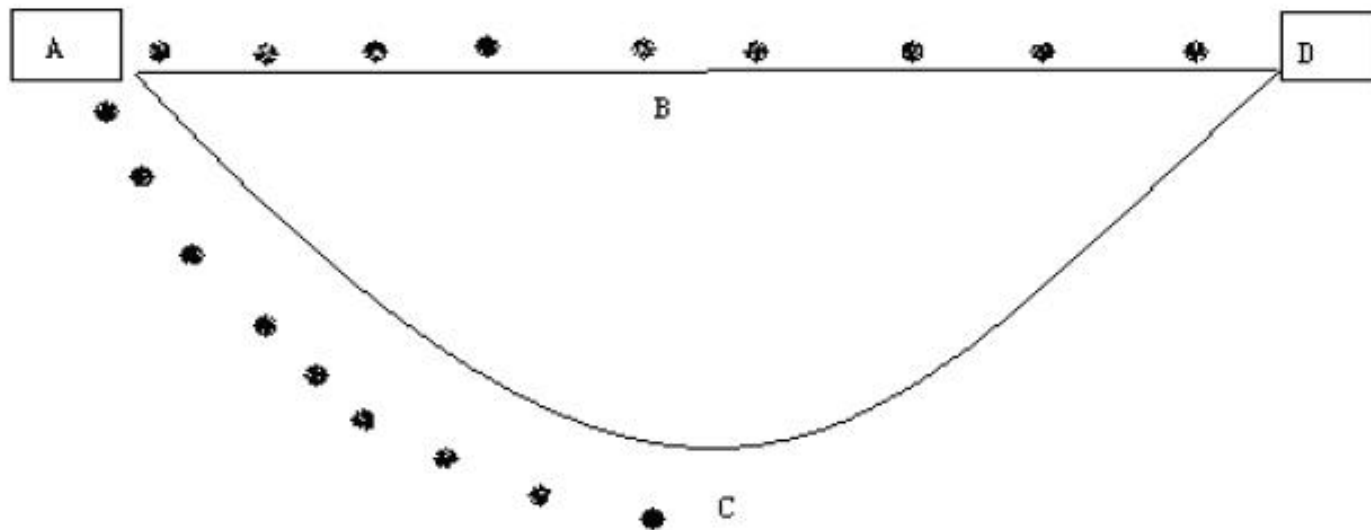
C、上方短路径蚂蚁到达决策点的时间早于选择下方长路径的蚂蚁

蚂蚁觅食原理



D、短路径上外激素以较高的速度积累
外激素多的短路径将吸收更多的蚂蚁，反过来，更多的蚂蚁在短路径上会留下更多的外激素，加上外激素挥发效应，最后，蚁群都选择了最短路径。

蚂蚁觅食原理

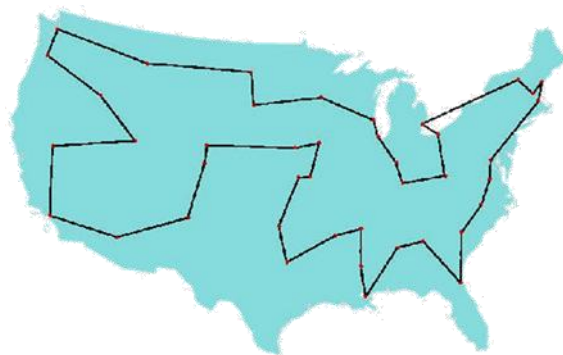


蚂蚁从A点出发，速度相同，食物在D点，可能随机选择路线ABD或ACD。假设初始时每条分配路线一只蚂蚁，每个时间单位行走一步，本图为经过9个时间单位时的情形：走ABD的蚂蚁到达终点，而走ACD的蚂蚁刚好走到C点，为一半路程。

蚂蚁觅食原理

- ❧ 假设蚂蚁每经过一处所留下的信息素为一个单位，则经过36个时间单位后，所有开始一起出发的蚂蚁都经过不同路径从D点取得了食物，此时ABD的路线往返了2趟，每一处的信息素为4个单位，而
- ❧ ACD的路线往返了一趟，每一处的信息素为2个单位，其比值为2: 1。
- ❧ 寻找食物的过程继续进行，则按信息素的指导，蚁群在ABD路线上增派一只蚂蚁（共2只），而ACD路线上仍然为一只蚂蚁。再经过36个时间单位后，两条线路上的信息素单位积累为12和4，比值为3: 1。
- ❧ 若继续进行，则按信息素的指导，最终所有的蚂蚁会放弃ACD路线，而都选择ABD路线。这也就是前面所提到的正反馈效应。

人工蚁群



- ❧ 基于以上蚁群寻找食物时的最优路径选择问题，可以构造人工蚁群，来解决最优化问题，如TSP问题。
- ❧ 人工蚁群中把具有简单功能的工作单元看作蚂蚁。二者的相似之处在于都是优先选择信息素浓度大的路径。较短路径的信息素浓度高，所以能够最终被所有蚂蚁选择，也就是最终的优化结果。
- ❧ 两者的区别在于人工蚁群有一定的记忆能力，能够记忆已经访问过的节点。同时，人工蚁群再选择下一条路径的时候是按一定算法规律有意识地寻找最短路径，而不是盲目的。例如在TSP问题中，可以预先知道当前城市到下一个目的地的距离。

ACO基本要素

```
graph TD; A([ACO基本要素]) --> B[路径构建]; A --> C[信息素更新];
```

路径构建

每只蚂蚁都随机选择一个城市作为其出发城市，并维护一个路径记忆向量，用来存放该蚂蚁依次经过的城市。蚂蚁在构建路径的每一步中，按照一个随机比例规则选择下一个要到达的城市。

信息素更新

当所有蚂蚁构建完路径后，算法将会对所有的路径进行全局信息素的更新。注意，我们所描述的是AS的ant-cycle版本，更新是在全部蚂蚁均完成了路径的构造后才进行的，信息素的浓度变化与蚂蚁在这一轮中构建的路径长度相关。

❧ 蚂蚁系统（Ant System, AS）是最基本的ACO算法，是以TSP作为应用实例提出的。

∞ 路径构建——

伪随机比例选择规则 (random proportional)

$$p_k(i, j) = \begin{cases} \frac{[\tau(i, j)]^\alpha [\eta(i, j)]^\beta}{\sum_{u \in J_k(i)} [\tau(i, u)]^\alpha [\eta(i, u)]^\beta}, & \text{if } j \in J_k(i) \\ 0, & \text{otherwise} \end{cases}$$

- ❖ 对于每只蚂蚁 k ，路径记忆向量 R^k 按照访问顺序记录了所有 k 已经经过的城市序号。设蚂蚁 k 当前所在城市为 i ，则其选择城市 j 作为下一个访问对象的概率如上式。 $J_k(i)$ 表示从城市 i 可以直接到达的、且又不在蚂蚁访问过的城市序列 R^k 中的城市集合。 $\eta(i, j)$ 是一个启发式信息，通常由 $\eta(i, j) = 1/d_{ij}$ 直接计算。 $\tau(i, j)$ 表示边 (i, j) 上的信息素量。

∞ 路径构建——

伪随机比例选择规则 (random proportional)

$$p_k(i, j) = \begin{cases} \frac{[\tau(i, j)]^\alpha [\eta(i, j)]^\beta}{\sum_{u \in J_k(i)} [\tau(i, u)]^\alpha [\eta(i, u)]^\beta}, & \text{if } j \in J_k(i) \\ 0, & \text{otherwise} \end{cases}$$

- ❖ 长度越短、信息素浓度越大的路径被蚂蚁选择的概率越大。 α 和 β 是两个预先设置的参数，用来控制启发式信息与信息素浓度作用的权重关系。当 $\alpha=0$ 时，算法演变成传统的随机贪心算法，最邻近城市被选中的概率最大。当 $\beta=0$ 时，蚂蚁完全只根据信息素浓度确定路径，算法将快速收敛，这样构建出的最优路径往往与实际目标有着较大的差异，算法的性能比较糟糕。

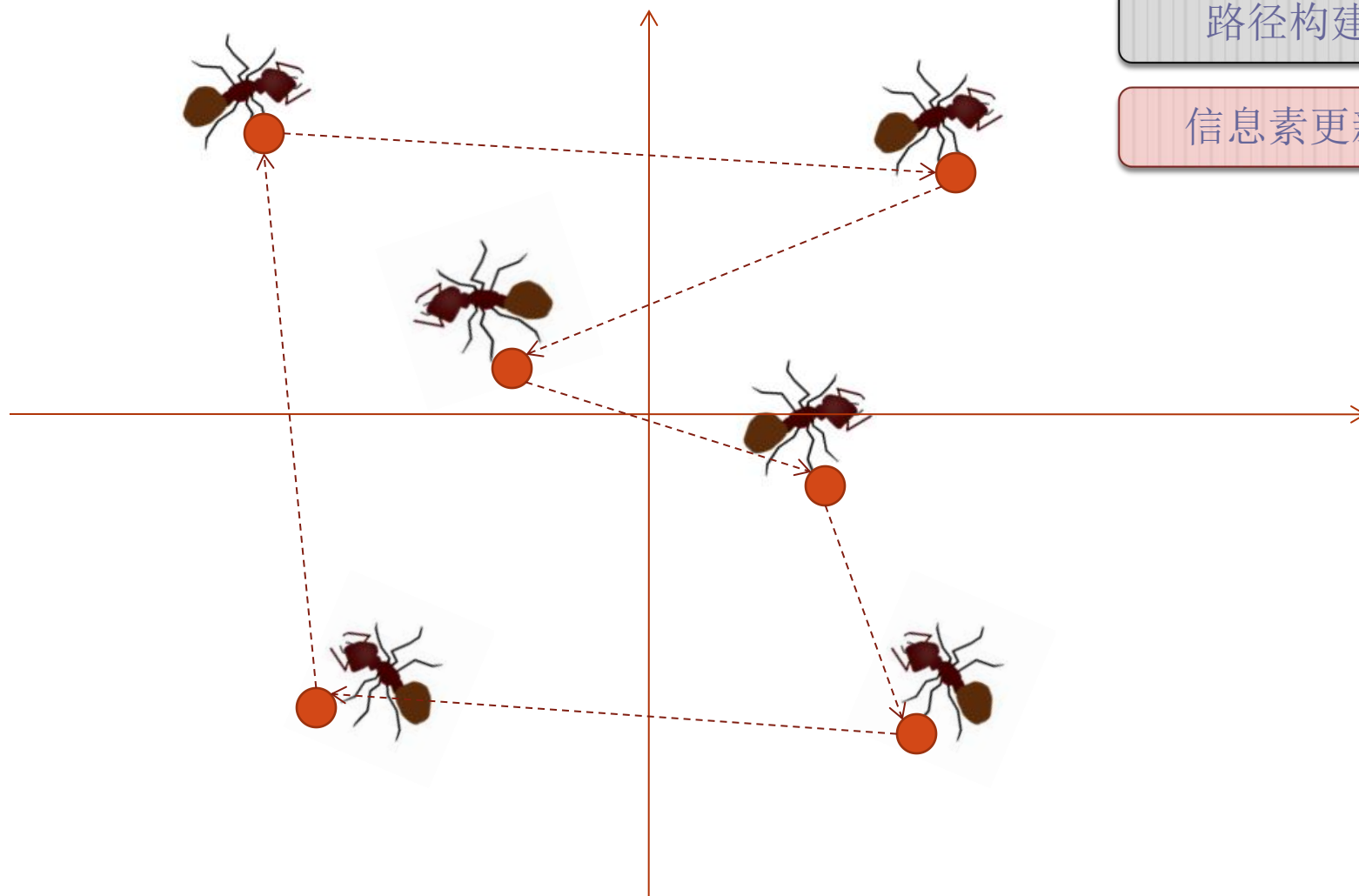
∞ 信息素更新

- (1) 在算法初始化时，问题空间中所有的边上的信息素都被初始化为 τ_0 。
- (2) 算法迭代每一轮，问题空间中的所有路径上的信息素都会发生蒸发，为所有边上的信息素乘上一个小于1的常数。信息素蒸发是自然界本身固有的特征，在算法中能够帮助避免信息素的无限积累，使得算法可以快速丢弃之前构建过的较差的路径。
- (3) 蚂蚁根据自己构建的路径长度在它们本轮经过的边上释放信息素。蚂蚁构建的路径越短、释放的信息素就越多。一条边被蚂蚁爬过的次数越多、它所获得的信息素也越多。
- (4) 迭代(2)，直至算法终止。

信息素更新

$$\tau(i, j) = (1 - \rho) \cdot \tau(i, j) + \sum_{k=1}^m \Delta \tau_k(i, j),$$
$$\Delta \tau_k(i, j) = \begin{cases} (C_k)^{-1}, & \text{if } (i, j) \in R^k \\ 0, & \text{otherwise} \end{cases}$$

- ❖ m 是蚂蚁个数； ρ 是信息素的蒸发率，规定 $0 < \rho \leq 1$ 。 $\Delta \tau_k(i, j)$ 是第 k 只蚂蚁在它经过的边上释放的信息素量，它等于蚂蚁 k 本轮构建路径长度的倒数。 C_k 表示路径长度，它是 R^k 中所有边的长度和。



路径构建

信息素更新

- ❖ 例 给出用蚁群算法求解一个四城市的TSP问题的执行步骤，四个城市A、B、C、D之间的距离矩阵如下

$$W = d_{ij} = \begin{bmatrix} \infty & 3 & 1 & 2 \\ 3 & \infty & 5 & 4 \\ 1 & 5 & \infty & 2 \\ 2 & 4 & 2 & \infty \end{bmatrix}$$

- ❖ 假设蚂蚁种群的规模 $m=3$,参数 $\alpha=1$, $\beta=2$, $\rho=0.5$ 。

步骤1：初始化。首先使用贪心算法得到路径(ACDBA)，则 $C^{nn}=f(ACDBA)=1+2+4+3=10$ 。

求得 $\tau_0=m/C^{nn}=3/10=0.3$ 。初始化所有边上的信息素 $\tau_{ij}=\tau_0$ 。

❖ 解:

步骤2.1: 为每只蚂蚁随机选择出发城市, 假设蚂蚁1选择城市A, 蚂蚁2选择城市B, 蚂蚁3选择城市D。

步骤2.2: 为每只蚂蚁选择下城市。我们仅以蚂蚁1为例, 当前城市 $i=A$, 可访问城市集合 $J_1(i)=\{B, C, D\}$ 。计算蚂蚁1选择B,C,D作为下一访问城市的概率:

$$A \Rightarrow \begin{cases} B: \tau_{AB}^{\alpha} \times \eta_{AB}^{\beta} = 0.3^1 \times (1/3)^2 = 0.033 & p(B) = 0.033 / (0.033 + 0.3 + 0.075) = 0.081 \\ C: \tau_{AC}^{\alpha} \times \eta_{AC}^{\beta} = 0.3^1 \times (1/1)^2 = 0.3 & p(C) = 0.3 / (0.033 + 0.3 + 0.075) = 0.74 \\ D: \tau_{AD}^{\alpha} \times \eta_{AD}^{\beta} = 0.3^1 \times (1/2)^2 = 0.075 & p(D) = 0.075 / (0.033 + 0.3 + 0.075) = 0.18 \end{cases}$$

用轮盘赌法则选择下城市。假设产生的随机数 $q=\text{random}(0,1)=0.05$, 则蚂蚁1将会选择城市B。

用同样的方法为蚂蚁2和3选择下一访问城市, 假设蚂蚁2选择城市D, 蚂蚁3选择城市A。

❖ 解:

步骤2.3: 当前蚂蚁1所在城市 $i=B$,路径记忆向量 $R^1=(AB)$, 可访问城市集合 $J_1(i)=\{C, D\}$ 。计算蚂蚁1选择C,D作为下一城市的概率:

$$B \Rightarrow \begin{cases} C: \tau_{BC}^{\alpha} \times \eta_{BC}^{\beta} = 0.3^1 \times (1/5)^2 = 0.012 \\ D: \tau_{BD}^{\alpha} \times \eta_{BD}^{\beta} = 0.3^1 \times (1/4)^2 = 0.019 \end{cases}$$

$$p(C) = 0.012 / (0.012 + 0.019) = 0.39$$

$$p(D) = 0.019 / (0.012 + 0.019) = 0.61$$

用轮盘赌法则选择下城市。假设产生的随机数 $q=\text{random}(0,1)=0.67$, 则蚂蚁1将会选择城市D。

用同样的方法为蚂蚁2和3选择下一访问城市, 假设蚂蚁2选择城市C, 蚂蚁3选择城市C。

❖ 解:

步骤2.4: 实际上此时路径已经构造完毕, 蚂蚁1构建的路径为(*ABDCA*)。蚂蚁2构建的路径为(*BDCAB*)。蚂蚁3构建的路径为(*DACBD*)。

$$W = d_{ij} = \begin{bmatrix} \infty & 3 & 1 & 2 \\ 3 & \infty & 5 & 4 \\ 1 & 5 & \infty & 2 \\ 2 & 4 & 2 & \infty \end{bmatrix}$$

❖ 解:

步骤3: 信息素更新。

计算每只蚂蚁构建的路径长度: $C_1=3+4+2+1=10$,
 $C_2=4+2+1+3=10$, $C_3=2+1+5+4=12$ 。更新每条边上的
信息素:

$$\tau_{AB} = (1 - \rho) \times \tau_{AB} + \sum_{k=1}^3 \Delta \tau_{AB}^k = 0.5 \times 0.3 + (1/10 + 1/10) = 0.35$$
$$\tau_{AC} = (1 - \rho) \times \tau_{AC} + \sum_{k=1}^3 \Delta \tau_{AC}^k = 0.5 \times 0.3 + (1/12) = 0.16$$

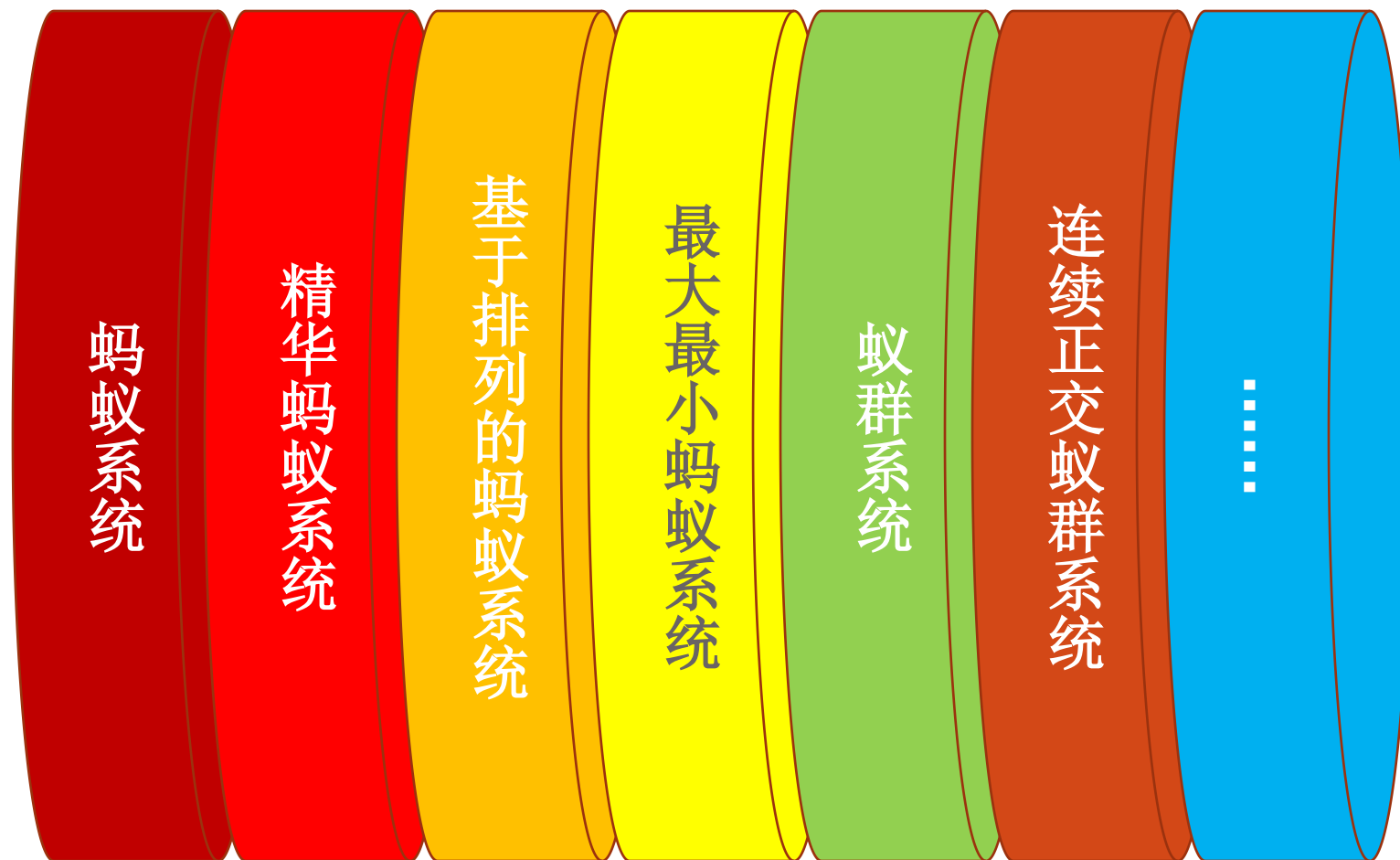
.....

如此, 根据公式依次计算出问题空间内所有边更新后的信息素量。

步骤4:

如果满足结束条件, 则输出全局最优结果并结束程序, 否则, 转向步骤2.1继续执行。

改进版本



1 精华蚂蚁系统

❧ 精华蚂蚁系统（Elitist Ant System, EAS）是对基础AS的第一次改进，它在原AS信息素更新原则的基础上增加了一个对至今最优路径的强化手段。

$$\tau(i, j) = (1 - \rho) \cdot \tau(i, j) + \sum_{k=1}^m \Delta\tau_k(i, j) + e\Delta\tau_b(i, j)$$
$$\Delta\tau_k(i, j) = \begin{cases} (C_k)^{-1}, & \text{if } (i, j) \in R^k \\ 0, & \text{otherwise} \end{cases} \quad \Delta\tau_b(i, j) = \begin{cases} (C_b)^{-1}, & \text{if } (i, j) \text{ 在路径 } T_b \text{ 上} \\ 0, & \text{otherwise} \end{cases}$$

❖ 引入这种额外的信息素强化手段有助于更好地引导蚂蚁搜索的偏向，使算法更快收敛。

2 基于排列的蚂蚁系统

∞ 基于排列的蚂蚁系统 (rank-based Ant System, AS_{rank}) 在AS的基础上给蚂蚁要释放的信息素大小加上一个权值, 进一步加大各边信息素量的差异, 以指导搜索。在每一轮所有蚂蚁构建完路径后, 它们将按照所得路径的长短进行排名, 只有生成了至今最优路径的蚂蚁和排名在前($\omega-1$)的蚂蚁才被允许释放信息素, 蚂蚁在边(i, j)上释放的信息素的权值由蚂蚁的排名决定。

$$\tau(i, j) = (1 - \rho) \cdot \tau(i, j) + \sum_{k=1}^{\omega-1} (\omega - k) \Delta\tau_k(i, j) + \omega \Delta\tau_b(i, j)$$

$$\Delta\tau_k(i, j) = \begin{cases} (C_k)^{-1}, & \text{if } (i, j) \in R^k \\ 0, & \text{otherwise} \end{cases} \quad \Delta\tau_b(i, j) = \begin{cases} (C_b)^{-1}, & \text{if } (i, j) \text{ 在路径 } T_b \text{ 上} \\ 0, & \text{otherwise} \end{cases}$$

❖ 权值($\omega-k$)对不同路径的信息素浓度差异起到了一个放大的作用, **AS_{rank}** 能更有力度地指导蚂蚁搜索。

3 最大最小蚂蚁系统

∞ 最大最小蚂蚁系统（MAX-MIN Ant System, MMAS）在基本AS算法的基础上进行了四项改进：

- (1) 只允许迭代最优蚂蚁（在本次迭代构建出最短路径的蚂蚁），或者至今最优蚂蚁释放信息素。
- (2) 信息素量大小的取值范围被限制在一个区间内。
- (3) 信息素初始值为信息素取值区间的上限，并伴随一个较小的信息素蒸发速率。
- (4) 每当系统进入停滞状态，问题空间内所有边上的信息素量都会被重新初始化。

3 最大最小蚂蚁系统

❧ 最大最小蚂蚁系统（MAX-MIN Ant System, MMAS）在基本AS算法的基础上进行了四项改进：

(1) 只允许迭代最优蚂蚁（在本次迭代构建出最短路径的蚂蚁），或者至今最优蚂蚁释放信息素。（迭代最优更新规则和至今最优更新规则在MMAS中会被交替使用。）

❖ 如果只使用至今最优更新规则进行信息素的更新，搜索的导向性很强，算法会很快收敛到 T_0 附近；反之，如果只使用迭代最优更新规则，则算法的探索能力会得到增强，但收敛速度会下降。实验结果表明，对于小规模의TSP问题，仅仅使用迭代最优信息素更新方式即可。随着问题规模的增大，至今最优信息素规则的使用变得越来越重要。

3 最大最小蚂蚁系统

❧ 最大最小蚂蚁系统（MAX-MIN Ant System, MMAS）在基本AS算法的基础上进行了四项改进：

(2) 信息素量大小的取值范围被限制在一个区间 $[\tau_{\min}, \tau_{\max}]$

❖ 内。当信息素浓度也被限制在一个范围内以后，位于城市 i 的蚂蚁 k 选择城市 j 作为下一城市的概率也将被限制在一个区间内。算法有效避免了陷入停滞状态（所有蚂蚁不断重复搜索同一条路径）的可能性。

(3) 信息素初始值为信息素取值区间的上限，并伴随一个较小的信息素蒸发速率。

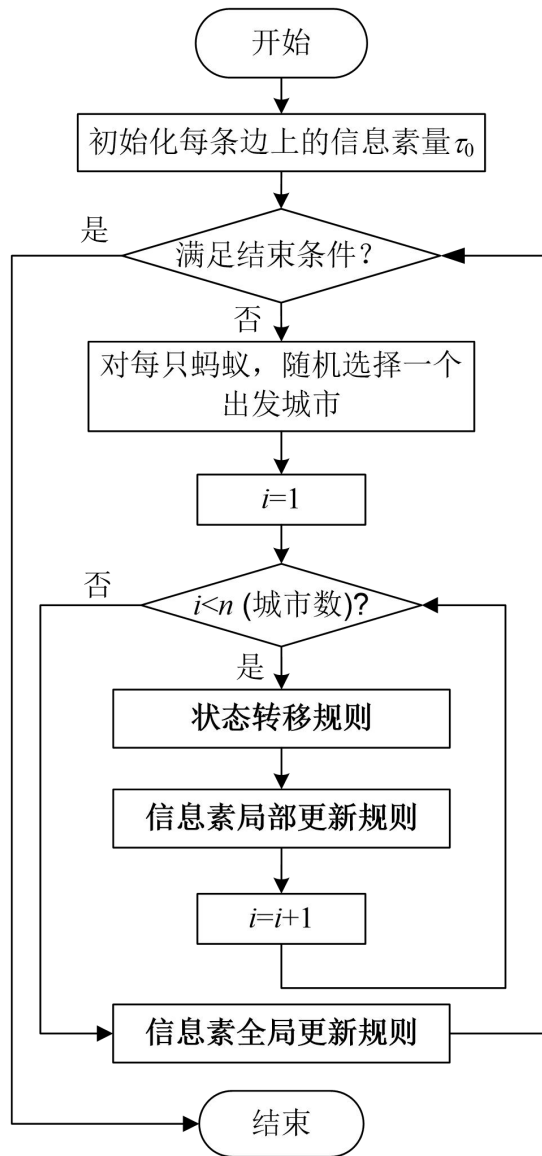
❖ 增强算法在初始阶段的探索能力，有助于蚂蚁“视野开阔地”进行全局范围内的搜索。

❖ 随后蚂蚁逐渐缩小搜索范围。

3 最大最小蚂蚁系统

- ❧ 最大最小蚂蚁系统（MAX-MIN Ant System, MMAS）在基本AS算法的基础上进行了四项改进：
 - (4) 每当系统进入停滞状态，问题空间内所有边上的信息素量都会被重新初始化。（我们通常通过对各条边上信息素量大小的统计或是观察算法在指定次数的迭代内至今最优路径有无被更新来判断算法是否停滞。）
- ❖ 有效地利用系统进入停滞状态后的迭代周期继续进行搜索，使算法具有更强的全局寻优能力。

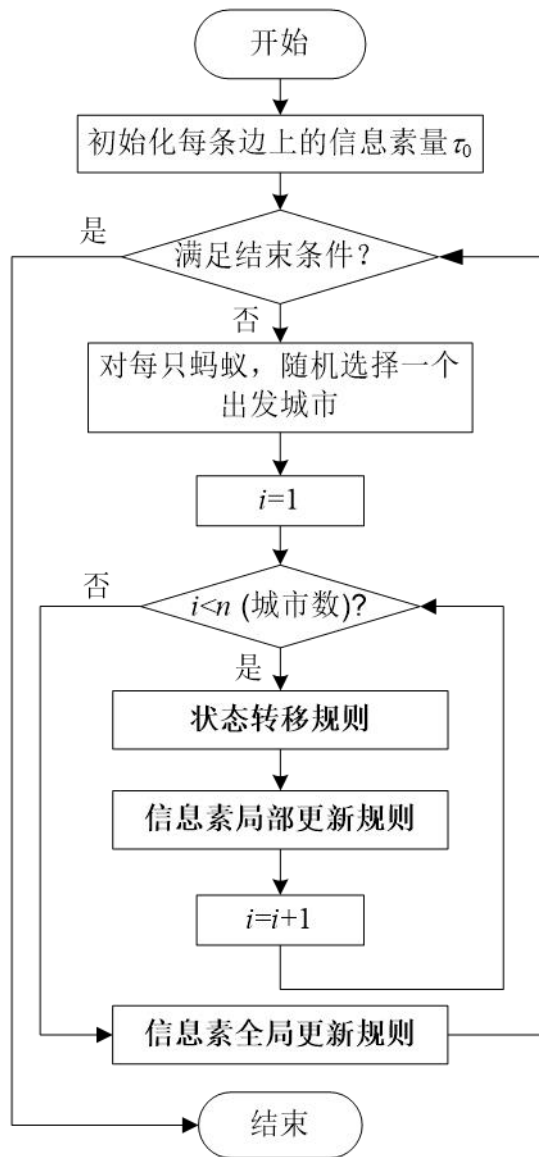
4 蚁群系统



✎ 1997年, 蚁群算法的创始人 Dorigo 在 “Ant colony system: a cooperative learning approach to the traveling salesman problem” 一文中提出了一种具有全新机制的ACO算法——蚁群系统 (Ant Colony System, ACS), 进一步提高了ACO算法的性能。

✎ ACS是蚁群算法发展史上的一个里程碑式的作品

4 蚁群系统

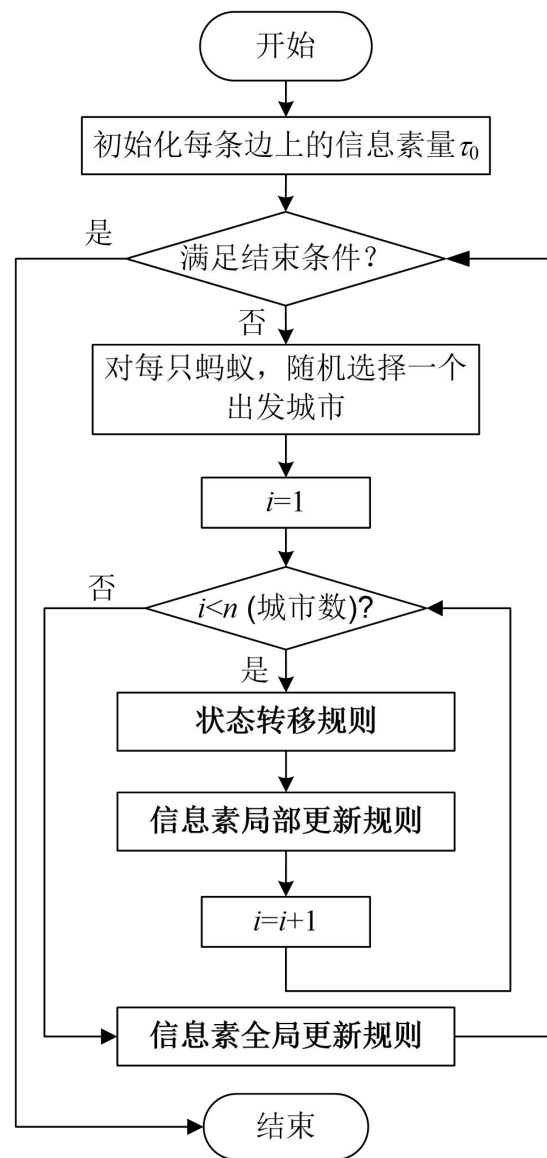


- ∞ (1) 使用一种伪随机比例规则 (pseudorandom proportional) 选择下城市节点，建立开发当前路径与探索新路径之间的平衡。

$$j = \begin{cases} \operatorname{argmax}_{j \in J_k(i)} \{ [\tau(i, j)]^\alpha [\eta(i, j)]^\beta \}, & \text{if } q \leq q_0 \\ S, & \text{otherwise} \end{cases}$$

- ∞ q_0 是一个 $[0, 1]$ 区间内的参数，当产生的随机数 $q \leq q_0$ 时，蚂蚁直接选择使启发式信息与信息素量的指数乘积最大的下城市节点，通常称之为开发 (exploitation)；反之，当产生的随机数 $q > q_0$ 时 ACS 将和各种 AS 算法一样使用轮盘赌选择策略，我们称之为偏向探索 (bias exploration)。
- ∞ 通过调整 q_0 ，我们能有效调节“开发”与“探索”之间的平衡，以决定算法是集中开发最优路径附近的区域，还是探索其它的区域。

4 蚁群系统



(2) 使用信息素全局更新规则，每轮迭代中所有蚂蚁都已构建完路径后，在属于至今最优路径的边上蒸发和释放信息素。

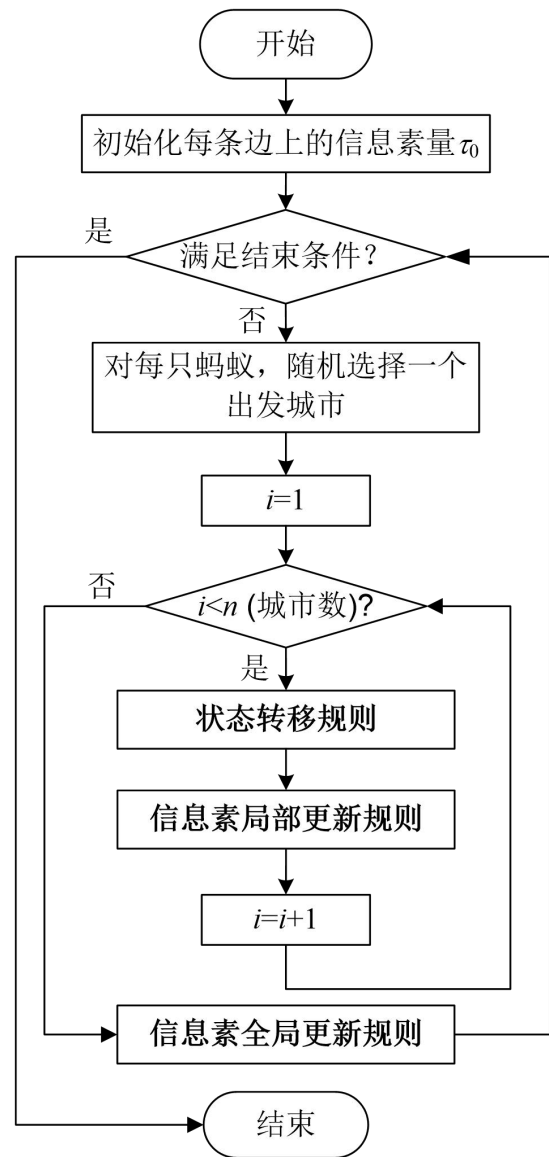
$$\tau(i, j) = (1 - \rho) \cdot \tau(i, j) + \rho \cdot \Delta\tau_b(i, j), \quad \forall (i, j) \in T_b$$

$$\Delta\tau_b(i, j) = 1/C_b$$

不论是信息素的蒸发还是释放，都只在属于至今最优路径的边上进行，这里与**AS**有很大的区别。因为**AS**算法将信息素的更新应用到了系统的所有边上，信息素更新的计算复杂度为 $O(n^2)$ ，而**ACS**算法的信息素更新计算复杂度降低为 $O(n)$ 。

参数 ρ 代表信息素蒸发的速率，新增加的信息素被乘上系数 ρ 后，更新后的信息素浓度被控制在旧信息素量与新释放的信息素量之间，用一种隐含的又更简单的方式实现了**MMAS**算法中对信息素量取值范围的限制。

4 蚁群系统

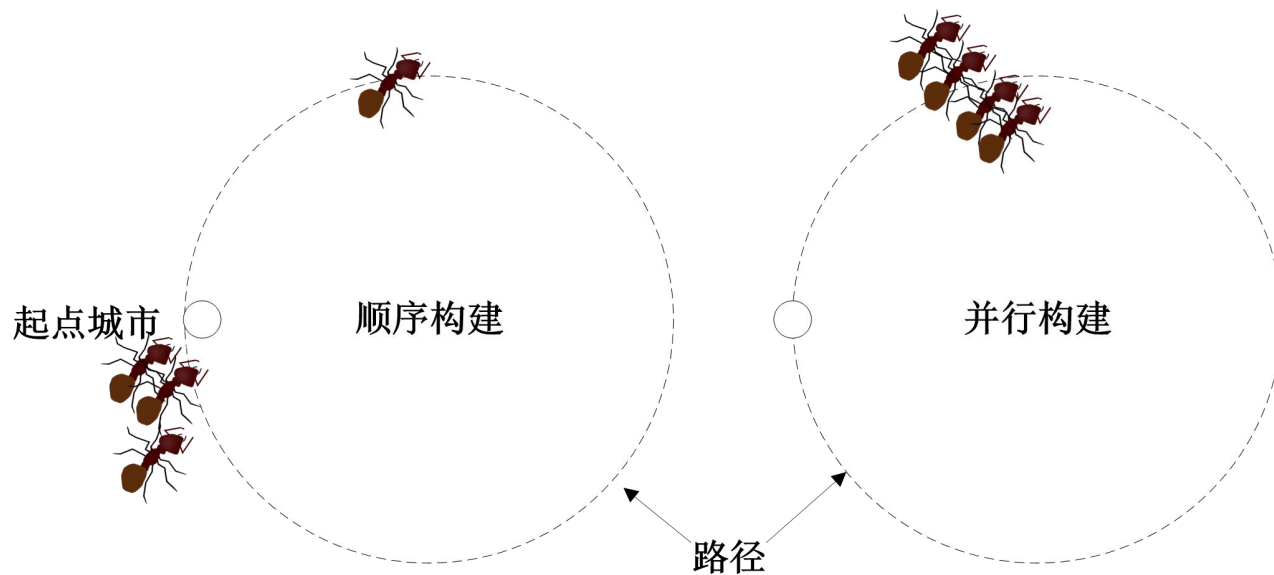


(3) 引入信息素局部更新规则，在路径构建过程中，对每一只蚂蚁，每当其经过一条边 (i, j) 时，它将立刻对这条边进行信息素的更新。

$$\tau(i, j) = (1 - \xi) \cdot \tau(i, j) + \xi \cdot \tau_0$$

信息素局部更新规则作用于某条边上会使得这条边被其他蚂蚁选中的概率减少。这种机制大大增加了算法的探索能力，后续蚂蚁倾向于探索未被使用过的边，有效地避免了算法进入停滞状态。

4 蚁群系统



顺序构建和**并行构建**。顺序构建是指当一只蚂蚁完成一轮完整的构建并返回到初始城市之后，下一只蚂蚁才开始构建；并行构建是指所有蚂蚁同时开始构建，每次所有蚂蚁各走一步（从当前城市移动到下一个城市）。对于**ACS**，要注意到两种路径构建方式会造成算法行为的区别。在**ACS**中通常我们选择让所有蚂蚁并行地工作。

5 连续正交蚁群系统

- ❧ 连续正交蚁群算法 (Continuous Orthogonal Ant Colony, COAC)：近年来，将应用领域扩展到连续空间的蚁群算法也在发展，连续正交蚁群就是其中比较优秀的一种。COAC通过在问题空间内自适应地选择和调整一定数量的区域，并利用蚂蚁在这些区域内进行正交搜索、在区域间进行状态转移、并更新各个区域的信息素来搜索问题空间中的最优解。
- ❧ COAC的基本思想是利用正交试验的方法将连续空间离散化。正交试验通过正交表来安排试验，既能使试验点分布得很均匀，又能减少试验次数。

相关应用

车间作业调度问题
(Job-Shop Scheduling Problem, JSP)

车辆路径问题
(Vehicle Routing Problem, VRP)

其他.....

图像处理

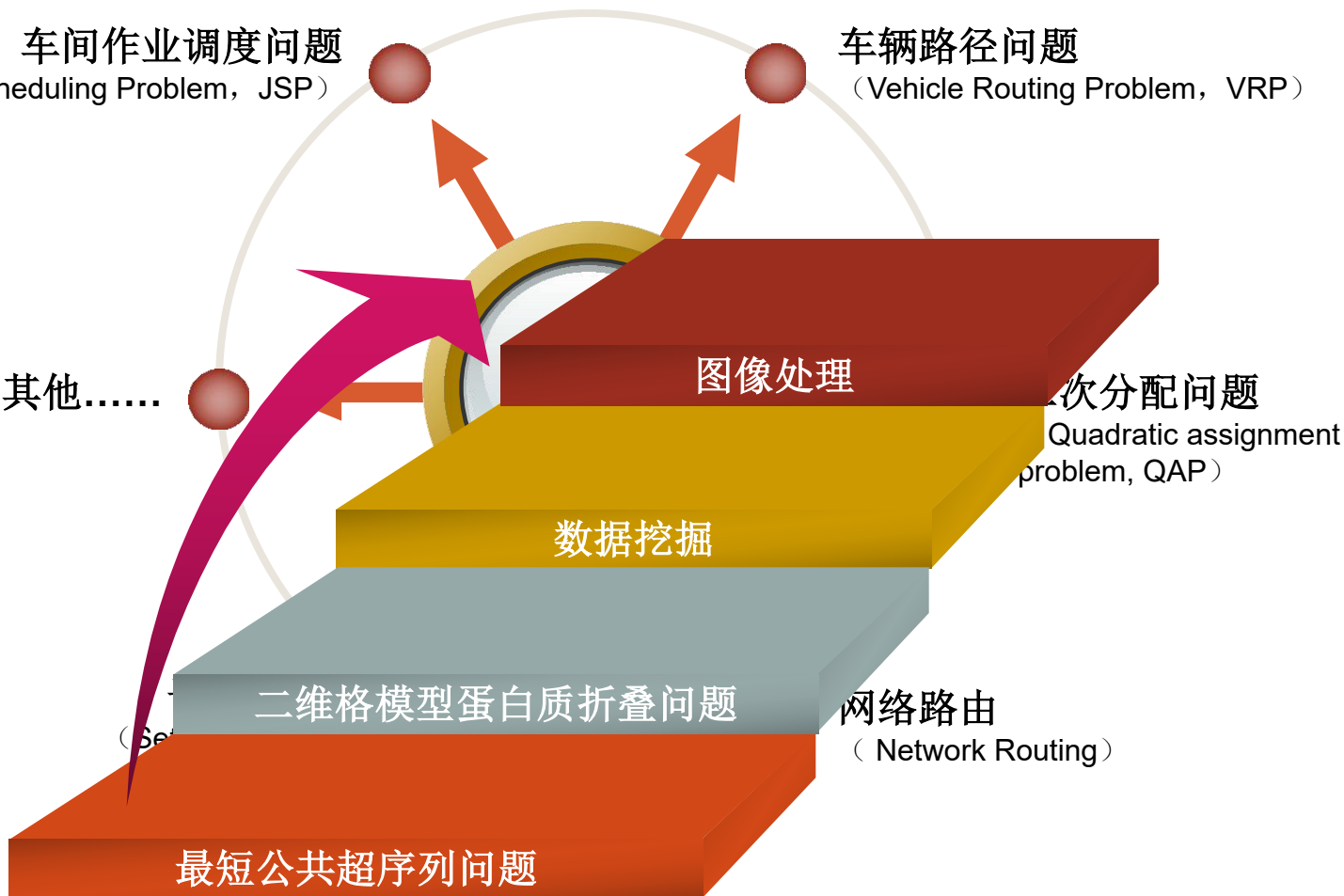
二次分配问题
(Quadratic assignment
problem, QAP)

数据挖掘

二维格模型蛋白质折叠问题
(Se...

网络路由
(Network Routing)

最短公共超序列问题



参数设置

参数	参考设置
蚂蚁数目 m	在用AS、EAS、AS _{rank} 和MMAS求解TSP问题时， m 取值等于城市数目 n 算法有较好性能；而对于ACS， $m=10$ 比较合适。
信息素权重 α 与启发式信息权重 β	在各类ACO算法中设置 $\alpha=1$ ， $\beta=2\sim 5$ 比较合适。
信息素挥发因子 ρ	对于AS和EAS， $\rho=0.5$ ；对于AS _{rank} ， $\rho=0.1$ ；对于MMAS， $\rho=0.02$ ；对于ACS， $\rho=0.1$ ，算法的综合性能较高。
初始信息素量 τ_0	对于AS， $\tau_0=m/C^{nn}$ ；对于EAS， $\tau_0=(e+m)/rC^{nn}$ ；对于AS _{rank} ， $\tau_0=0.5r(r-1)/rC^{nn}$ ；对于MMAS， $\tau_0=1/rC^{nn}$ ；对于ACS， $\tau_0=1/nC^{nn}$ 。
释放信息素的蚂蚁个数 ω	在AS _{rank} 中，参数 ω 设置为 $\omega=6$ 。
进化停滞判定代数 rs	在MMAS中，参数 rs 设置为 $rs=25$ 。
信息素局部挥发因子 ξ	在ACS中，参数 ξ 设置为 $\xi=0.1$ 。
伪随机因子 q_0	在ACS中，参数 q_0 设置为 $q_0=0.1$ 。

蚁群算法的特点

(1) 其原理是一种正反馈机制或称增强型学习系统，它通过“最优路径上蚂蚁数量的增加+信息素强度增加+后来蚂蚁选择概率增大+最优路径上蚂蚁数量更大增加”达到最终收敛于最优路径上。

(2) 它是一种通用型随机优化方法，吸收了蚂蚁的行为特性(内在搜索机制)。它是使用人工蚂蚁仿真(也称蚂蚁系统)来求解问题，但人工蚂蚁决不是对实际蚂蚁的一种简单模拟，它融进了人类的智能，人工蚂蚁有一定的记忆；人工蚂蚁不完全是瞎的；人工蚂蚁生活的时空是离散的。

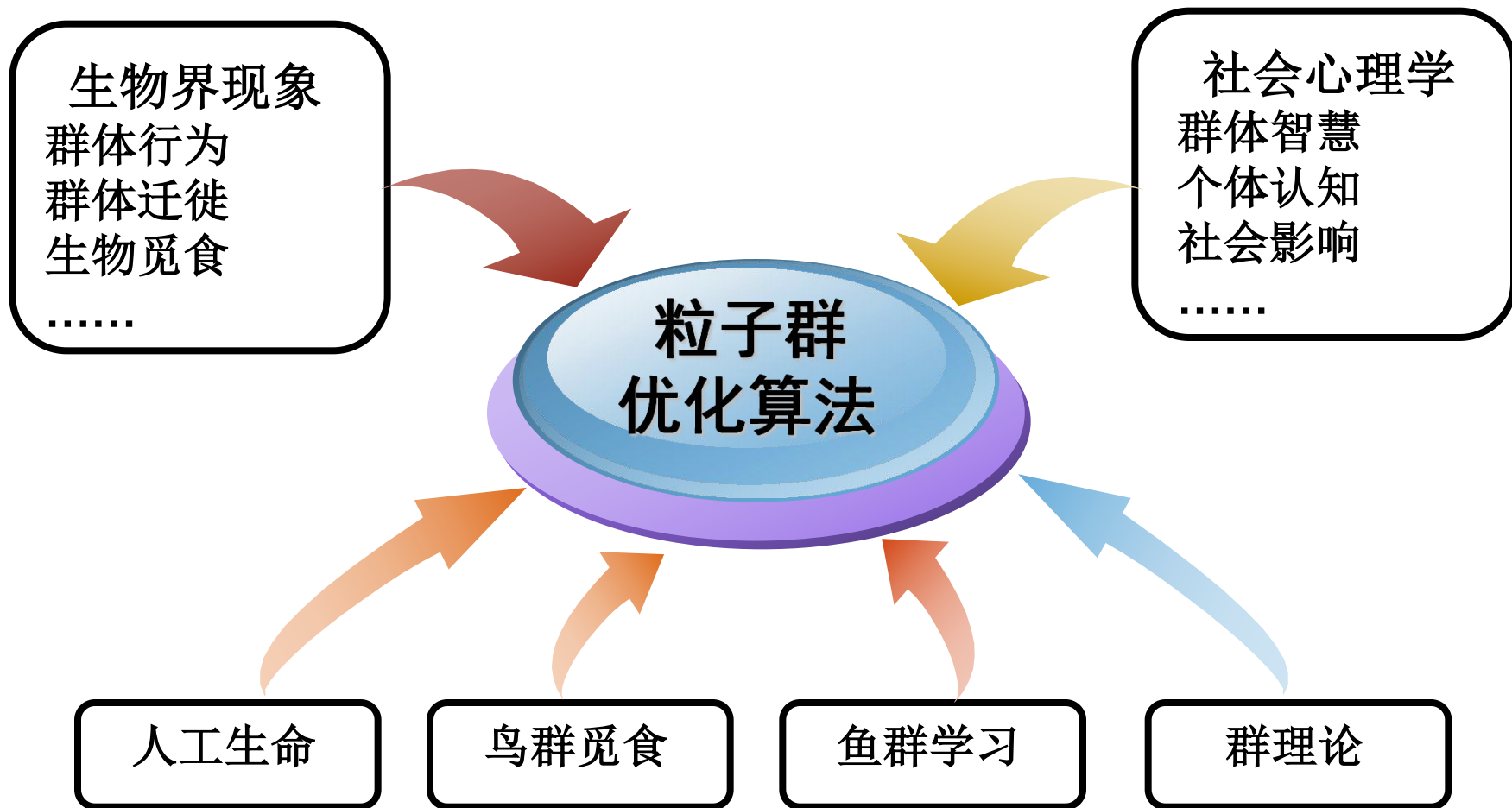
(3) 它是一种分布式的优化方法，不仅适合目前的串行计算机，而且适合未来的并行计算机。

(4) 它是一种全局优化的方法，不仅可用于求解单目标优化问题，而且可用于求解多目标优化问题。

(5) 它是一种启发式算法，计算复杂性为 $O(nc * m * n^2)$ ，其中 nc 是迭代次数， m 是蚂蚁数目， n 是目标的节点数目。

第三节：粒子群优化算法

思想来源



粒子群优化

粒子群优化 (Particle Swarm Optimization, 简称PSO)

[1], 又称为微粒群算法, 是由美国的心理学家J. Kennedy和电气工程师R. Eberhart于1995年提出的一种算法。

[1] J. Kennedy and R. Eberhart. “Particle Swarm Optimization,” Proceedings of IEEE International Conference on Neural Networks, Perth, WA, 1995, pp. 1942-1948.

PSO是模拟鸟群飞行觅食行为的一种随机优化算法。





Kennedy和Eberhart的初衷是研究鸟的觅食行为，建立一个模型来模拟鸟群寻找食源的现象。

Kennedy和Eberhart在实验中发现，这个模型有很强的优化能力。



鸟群觅食

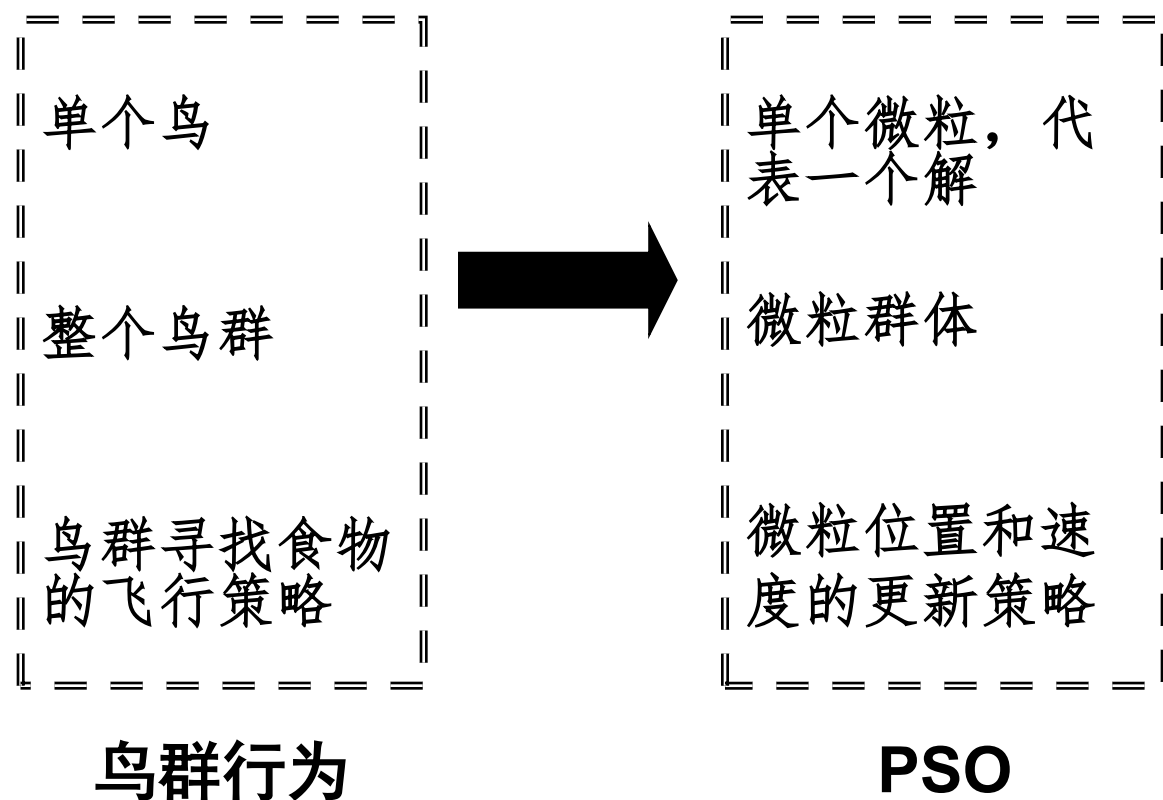
- ❧ 鸟群在行进中会突然同步的改变方向，散开或者聚集等。那么一定有某种潜在的能力或规则保证了这些同步的行为。
- ❧ 科学家都认为上述行为是基于不可预知的鸟类社会行为中的群体动态学。在这些早期的模型中仅仅依赖个体间距的操作，也就是说，这种同步是鸟群中个体之间努力保持最优的距离的结果。
- ❧ 最简单有效的方法就是搜寻目前离食物最近的鸟的周围区域。

PSO原理

- 将问题的搜索空间类比于鸟类的飞行空间。
- 将每一只鸟抽象为一个无质量无体积的微粒 (Particle)，用于表征每个候选解。
- 优化寻求最优解等同于鸟群寻找食物。
- PSO为每个微粒制定了类似于鸟类运动的简单行为规则，从而使整个微粒群的运动表现出与鸟类觅食类似的特性，用于求解复杂优化问题。

PSO原理

PSO是对鸟群寻找食物这种群体行为的模拟



PSO算法

鸟群怎样尽量快的找到食物？

一个有效的方法就是搜寻目前离食物最近同伴的位置。

每个粒子有一个位置和一个速度。

粒子通过跟踪两个“极值”来更新自己。第一个就是粒子本身所找到的最优解，即 $pBest$ ；另一个是整个种群的最优解，即 $gBest$ 。

PSO算法

假设搜索空间为 d 维，群体（Swarm）中的粒子数为 n 。

在第 t 次迭代中，群体中第 i 个粒子的位置表示为：

$$\vec{x}_i(t) = (x_{i1}, x_{i2}, \dots, x_{id})$$

第 i 个粒子在飞行中所经过的最佳位置为：

$$\vec{p}_i(t) = (p_{i1}, p_{i2}, \dots, p_{id})$$

群体中所有粒子的 $\vec{p}_i(t)$ 中最佳位置为：

$$\vec{p}_g(t) = (p_{g1}, p_{g2}, \dots, p_{gd})$$

第 i 个粒子的位置变化速度为：

$$\vec{v}_i(t) = (v_{i1}, v_{i2}, \dots, v_{id})$$

PSO算法

每个粒子的位置按如下公式变化：

$$v_{ij}(t+1) = v_{ij}(t) + c_1 r_1 (p_{ij}(t) - x_{ij}(t)) + c_2 r_2 (p_{gj}(t) - x_{ij}(t))$$

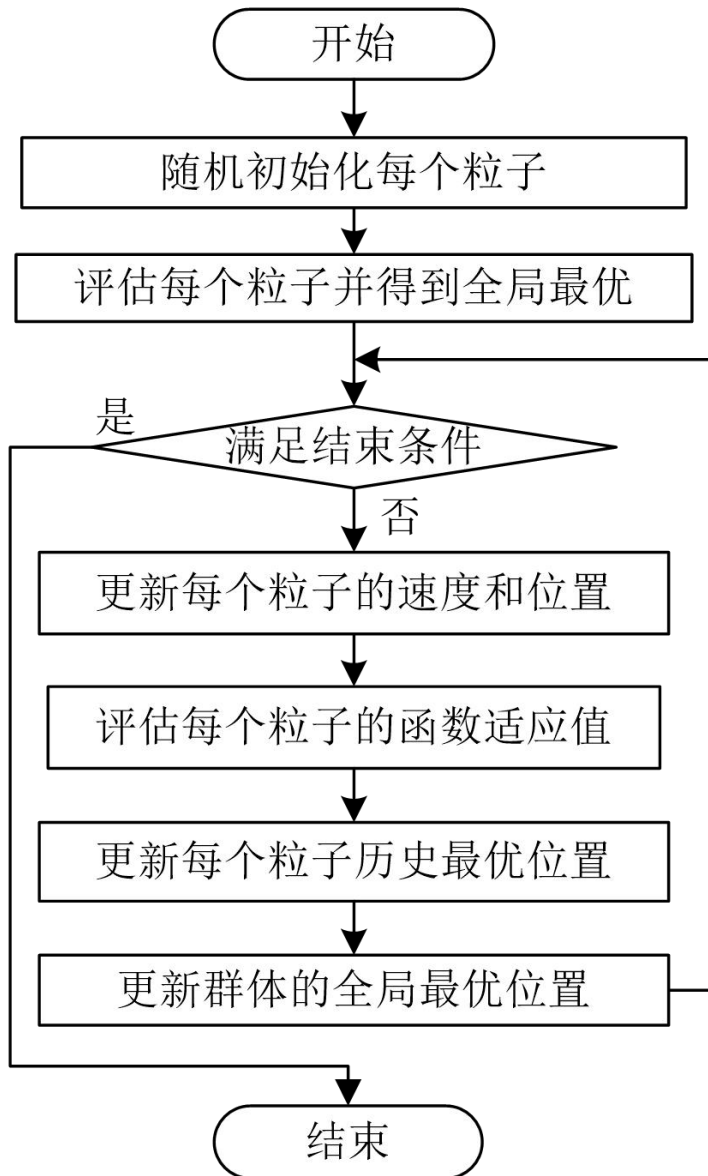
$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

其中 r_1 和 r_2 为[0,1]间的随机数； c_1 和 c_2 为加速系数，用来控制 $p_{ij}(t)$ 和 $p_{gj}(t)$ 对粒子飞行方向的影响。

若粒子 i 的位置的第 j 维超过范围 $[x_{\min}, x_{\max}]$ ，则取边界值。

若粒子 i 的速度的第 j 维超过范围 $[V_{\min}, V_{\max}]$ ，则取边界值。

PSO算法流程图和伪代码



//功能：粒子群优化算法伪代码

//说明：本例以求问题最小值为目标

//参数： N 为群体规模

procedure PSO

for each particle i

Initialize velocity V_i and position X_i for particle i

Evaluate particle i and set $pBest_i = X_i$

end for

$gBest = \min \{pBest_i\}$

while not stop

for $i=1$ to N

Update the velocity and position of particle i

Evaluate particle i

if $\text{fit}(X_i) < \text{fit}(pBest_i)$

$pBest_i = X_i;$

if $\text{fit}(pBest_i) < \text{fit}(gBest)$

$gBest = pBest_i;$

end for

end while

print $gBest$

end procedure

应用举例

例 已知函数 $y=f(x_1,x_2)=x_1^2+x_2^2$

， $-10 \leq x_1, x_2 \leq 10$

其中

群优化算法求解 y 的最小值。

， 用粒子

运行步骤

步骤1：初始化。

假设种群大小是 $N=3$ ；在搜索空间中随机初始化每个解的速度和位置，计算适应函数值，并且得到粒子的历史最优位置和群体的全局最优位置。

$$p_1 = \begin{cases} v_1 = (3, 2) \\ x_1 = (8, -5) \end{cases} \begin{cases} f_1 = 8^2 + (-5)^2 = 64 + 25 = 89 \\ \mathbf{pBest}_1 = \mathbf{x}_1 = (8, -5) \end{cases}$$

$$p_2 = \begin{cases} v_2 = (-3, -2) \\ x_2 = (-5, 9) \end{cases} \begin{cases} f_2 = (-5)^2 + 9^2 = 25 + 81 = 106 \\ \mathbf{pBest}_2 = \mathbf{x}_2 = (-5, 9) \end{cases}$$

$$p_3 = \begin{cases} v_3 = (5, 3) \\ x_3 = (-7, -8) \end{cases} \begin{cases} f_3 = (-7)^2 + (-8)^2 = 49 + 64 = 113 \\ \mathbf{pBest}_3 = \mathbf{x}_3 = (-7, -8) \end{cases}$$

$$\mathbf{gBest} = \mathbf{pBest}_1 = (8, -5)$$

运行步骤

步骤2: 粒子的速度和位置更新。

根据自身的历史最优位置和全局的最优位置，更新每个粒子的速度和位置。

$$p_1 = \begin{cases} v_1 = \omega \times v_1 + c_1 \times r_1 \times (pBest_1 - x_1) + c_2 \times r_2 \times (gBest - x_1) \\ \Rightarrow v_1 = \begin{cases} 0.5 \times 3 + 0 + 0 = 1.5 \\ 0.5 \times 2 + 0 + 0 = 1 \end{cases} = (1.5, 1) \\ x_1 = x_1 + v_1 = (8, -5) + (1.5, 1) = (9.5, -4) \end{cases}$$

$$p_2 = \begin{cases} v_2 = \omega \times v_2 + c_1 \times r_1 \times (pBest_2 - x_2) + c_2 \times r_2 \times (gBest - x_2) \\ \Rightarrow v_2 = \begin{cases} 0.5 \times (-3) + 0 + 2 \times 0.3 \times (8 - (-5)) = 6.1 \\ 0.5 \times (-2) + 0 + 2 \times 0.1 \times ((-5) - 9) = 1.8 \end{cases} = (6.1, 1.8) \\ x_1 = x_1 + v_1 = (-5, 9) + (6.1, 1.8) = (1.1, 10.8) = (1.1, 10) \end{cases}$$

注意!

对于越界的位置，需要进行合法性调整

$$p_3 = \begin{cases} v_3 = \omega \times v_3 + c_1 \times r_1 \times (pBest_3 - x_3) + c_2 \times r_2 \times (gBest - x_3) \\ \Rightarrow v_3 = \begin{cases} 0.5 \times 5 + 0 + 2 \times 0.05 \times (8 - (-7)) = 3.5 \\ 0.5 \times 3 + 0 + 2 \times 0.8 \times ((-5) - (-8)) = 6.3 \end{cases} = (3.5, 6.3) \\ x_1 = x_1 + v_1 = (-7, -8) + (3.5, 6.3) = (-3.5, -1.7) \end{cases}$$

w 是惯量权重，一般取 $[0,1]$ 区间的数，这里假设为0.5

c_1 和 c_2 为加速系数，通常取固定值2.0

r_1 和 r_2 是 $[0,1]$ 区间的随机数

运行步骤

步骤3：评估粒子的适应度函数值。
更新粒子的历史最优位置和全局的最优位置。

$$f_1^* = 9.5^2 + (-4)^2 = 90.25 + 16 = 106.25 > f_1 = 89$$

$$\begin{cases} f_1 = 89 \\ \mathbf{pBest}_1 = (8, -5) \end{cases}$$

$$f_2^* = 1.1^2 + 10^2 = 1.21 + 100 = 101.21 < 106 = f_2$$

$$\begin{cases} f_2 = f_2^* = 101.21 \\ \mathbf{pBest}_2 = X_2 = (1.1, 10) \end{cases}$$

$$f_3^* = (-3.5)^2 + (-1.7)^2 = 12.25 + 2.89 = 15.14 < 113 = f_3$$

$$\begin{cases} f_3 = f_3^* = 15.14 \\ \mathbf{pBest}_3 = \mathbf{x}_3 = (-3.5, -1.7) \end{cases}$$

$$\mathbf{gBest} = \mathbf{pBest}_3 = (-3.5, -1.7)$$

运行步骤

步骤4:

如果满足结束条件，则输出全局最优结果并结束程序，
否则，转向步骤2继续执行。

步骤1: 初始化。

假设种群大小是 $N=3$ ；在搜索空间中随机初始化每个解的速度和位置，计算适应函数值，并且得到粒子的历史最优位置和群体的全局最优位置。

$$p_1 = \begin{cases} v_1 = (3, 2) \\ x_1 = (8, -5) \end{cases} \begin{cases} f_1 = 8^2 + (-5)^2 = 64 + 25 = 89 \\ pBest_1 = x_1 = (8, -5) \end{cases}$$

$$p_2 = \begin{cases} v_2 = (-3, -2) \\ x_2 = (-5, 9) \end{cases} \begin{cases} f_2 = (-5)^2 + 9^2 = 25 + 81 = 106 \\ pBest_2 = x_2 = (-5, 9) \end{cases}$$

$$p_3 = \begin{cases} v_3 = (5, 3) \\ x_3 = (-7, -8) \end{cases} \begin{cases} f_3 = (-7)^2 + (-8)^2 = 49 + 64 = 113 \\ pBest_3 = x_3 = (-7, -8) \end{cases}$$

$$gBest = pBest_1 = (8, -5)$$

步骤3: 评估粒子的适应度函数值。

更新粒子的历史最优位置和全局的最优位置。

$$f_1^* = 9.5^2 + (-4)^2 = 90.25 + 16 = 106.25 > f_1 = 89$$

$$\begin{cases} f_1 = 89 \\ pBest_1 = (8, -5) \end{cases}$$

$$f_2^* = 1.1^2 + 10^2 = 1.21 + 100 = 101.21 < 106 = f_2$$

$$\begin{cases} f_2 = f_2^* = 101.21 \\ pBest_2 = X_2 = (1.1, 10) \end{cases}$$

$$f_3^* = (-3.5)^2 + (-1.7)^2 = 12.25 + 2.89 = 15.14 < 113 = f_3$$

$$\begin{cases} f_3 = f_3^* = 15.14 \\ pBest_3 = x_3 = (-3.5, -1.7) \end{cases}$$

$$gBest = pBest_3 = (-3.5, -1.7)$$

步骤2: 粒子的速度和位置更新。

根据自身的历史最优位置和全局的最优位置，更新每个粒子的速度和位置。

$$p_1 = \begin{cases} v_1 = \omega \times v_1 + c_1 \times r_1 \times (pBest_1 - x_1) + c_2 \times r_2 \times (gBest - x_1) \\ \Rightarrow v_1 = \begin{cases} 0.5 \times 3 + 0 + 0 = 1.5 \\ 0.5 \times 2 + 0 + 0 = 1 \end{cases} = (1.5, 1) \\ x_1 = x_1 + v_1 = (8, -5) + (1.5, 1) = (9.5, -4) \end{cases}$$

$$p_2 = \begin{cases} v_2 = \omega \times v_2 + c_1 \times r_1 \times (pBest_2 - x_2) + c_2 \times r_2 \times (gBest - x_2) \\ \Rightarrow v_2 = \begin{cases} 0.5 \times (-3) + 0 + 2 \times 0.3 \times (8 - (-5)) = 6.1 \\ 0.5 \times (-2) + 0 + 2 \times 0.1 \times ((-5) - 9) = 1.8 \end{cases} = (6.1, 1.8) \\ x_1 = x_1 + v_1 = (-5, 9) + (6.1, 1.8) = (1.1, 10.8) = (1.1, 10) \end{cases}$$

注意!

对于越界的位置，需要进行合法性调整

$$p_3 = \begin{cases} v_3 = \omega \times v_3 + c_1 \times r_1 \times (pBest_3 - x_3) + c_2 \times r_2 \times (gBest - x_3) \\ \Rightarrow v_3 = \begin{cases} 0.5 \times 5 + 0 + 2 \times 0.05 \times (8 - (-7)) = 3.5 \\ 0.5 \times 3 + 0 + 2 \times 0.8 \times ((-5) - (-8)) = 6.3 \end{cases} = (3.5, 6.3) \\ x_1 = x_1 + v_1 = (-7, -8) + (3.5, 6.3) = (-3.5, -1.7) \end{cases}$$

w 是惯量权重，一般取 $[0, 1]$ 区间的数，这里假设为0.5
 c_1 和 c_2 为加速系数，通常取固定值2.0
 r_1 和 r_2 是 $[0, 1]$ 区间的随机数

步骤4: 如果满足结束条件，则输出全局最优结果并结束程序，否则，转向步骤2继续执行。

PSO更新公式



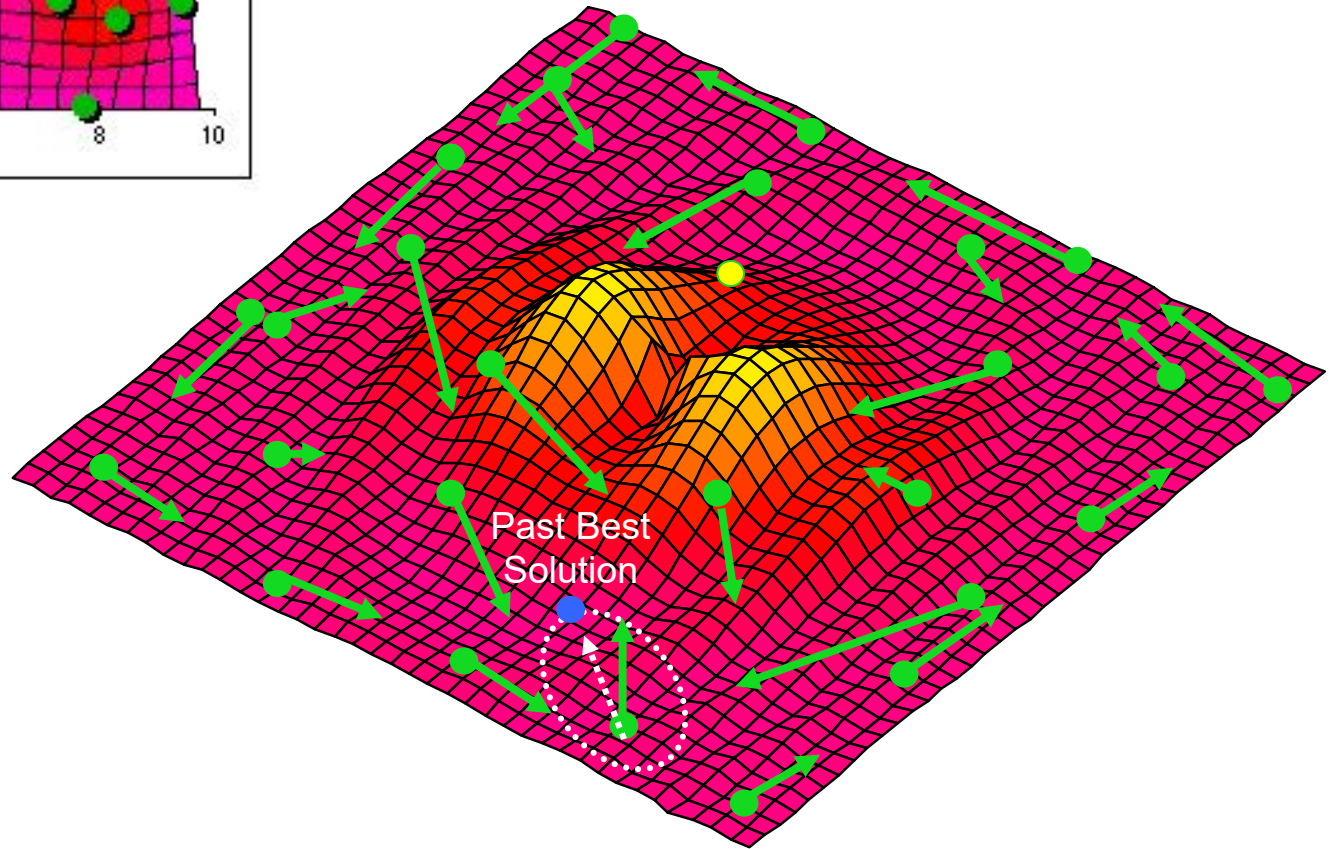
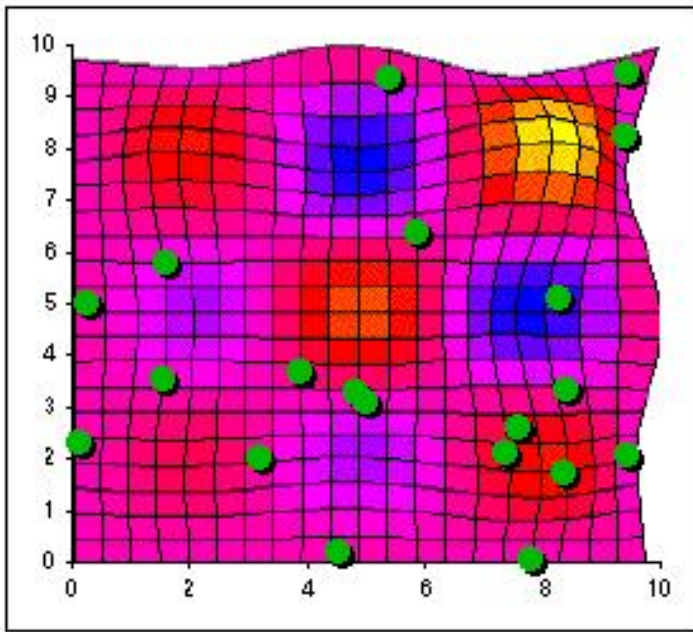
速度冲量 认知项 社会项

$$v_{ij}(t+1) = v_{ij}(t) + \boxed{c_1 r_1 (p_{ij}(t) - x_{ij}(t))} + \boxed{c_2 r_2 (p_{gj}(t) - x_{ij}(t))}$$
$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

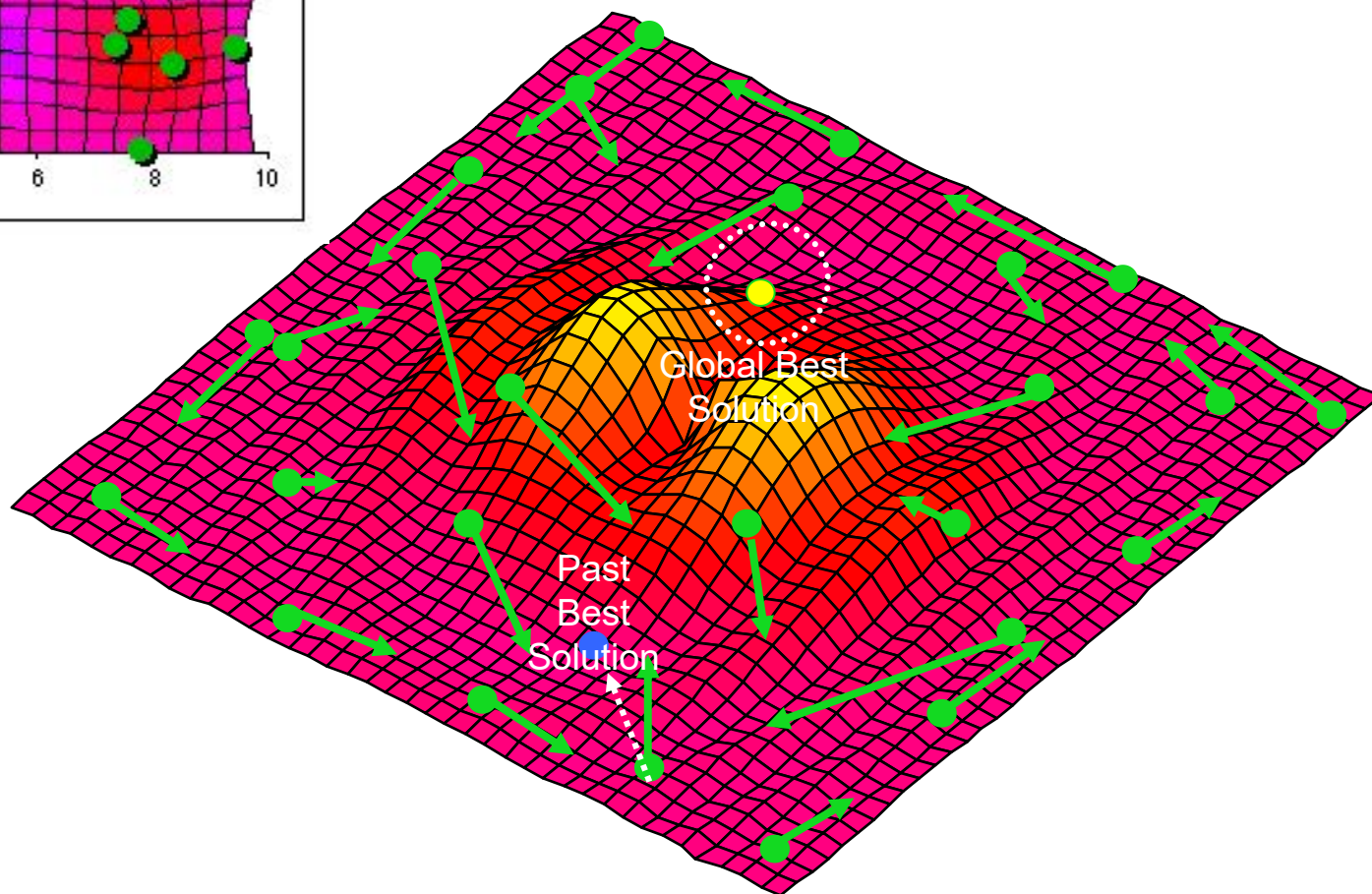
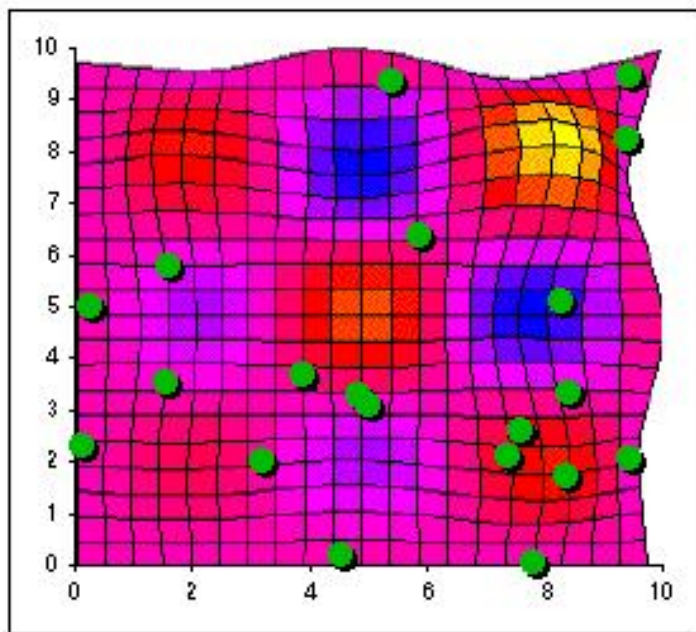
微粒速度更新公式包括三项:

- ◆速度冲量：指引微粒继续飞行的先前微粒速度。
 - ◆认知项：微粒重新返回其所经过的最好位置的趋势，既微粒本身记忆的影响。
 - ◆社会项：微粒被当前最好位置吸引的趋势，既群体信息的影响。
- 在三部分的共同作用下，微粒根据历史经验并利用全局信息，不断调整自己的位置，以期找到最优解。

认知项



社会项



PSO算法

惯性权重

速度冲量导致微粒按照先前速度方向继续移动。Yuhui Shi [1] 提出一个惯性权重 w 来控制先前微粒速度的影响。

惯性权重

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(p_{ij}(t) - x_{ij}(t)) + c_2r_2(p_{gj}(t) - x_{ij}(t))$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

[1] Y. Shi, R. Eberhart. "A modified particle swarm optimizer," Proceedings of IEEE World Congress on Computational Intelligence, Anchorage, AK, 1998, pp. 69-73.

PSO算法

通过调节 w 值，可以控制PSO的全局探索和局部开发能力：

- $w \geq 1$ ：微粒速度随迭代次数的增加而增加，微粒发散。
- $0 < w < 1$ ：微粒减速，算法的收敛性依靠惯性权重 c_1 和 c_2 。
- $w < 0$ ：微粒速度随迭代次数的增加而减小，最后趋近0，算法收敛。

实验表明， $w=0.7298$ 和 $c_1 = c_2 = 1.497$ 时算法具有较好的收敛性能。

自适应或动态惯性权重值：

- Eberhart和Shi： w 的线性递减策略；
- Shi和Eberhart：以当前惯性权重值和当前算法最优值为模糊系统的输入，给出一种惯性权重的模糊控制方法。

加速度系数

加速度系数

加速度系数

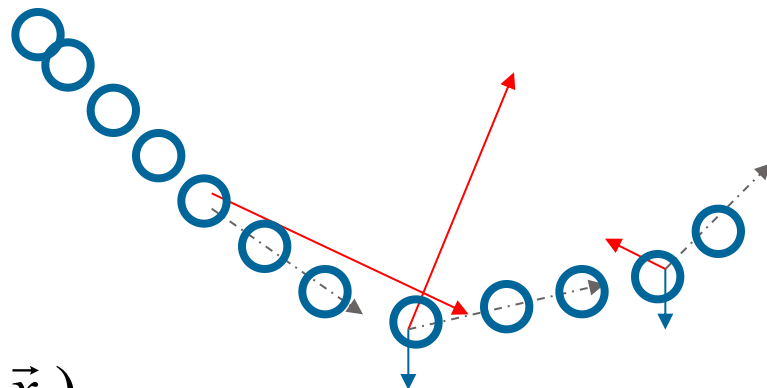
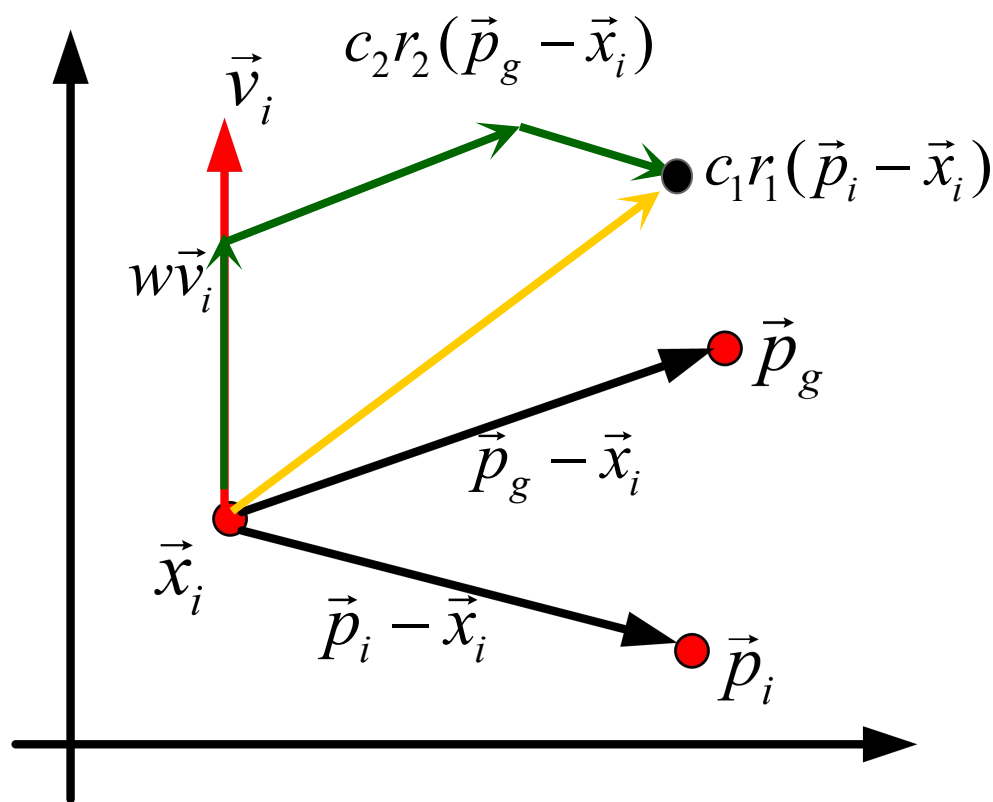
$$v_{ij}(t+1) = wv_{ij}(t) + c_1 r_1 (p_{ij}(t) - x_{ij}(t)) + c_2 r_2 (p_{gj}(t) - x_{ij}(t))$$
$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

加速度系数，又称学习因子，用来控制 $\vec{p}_i$ 和 $\vec{p}_g$ 对微粒飞行方向的影响。

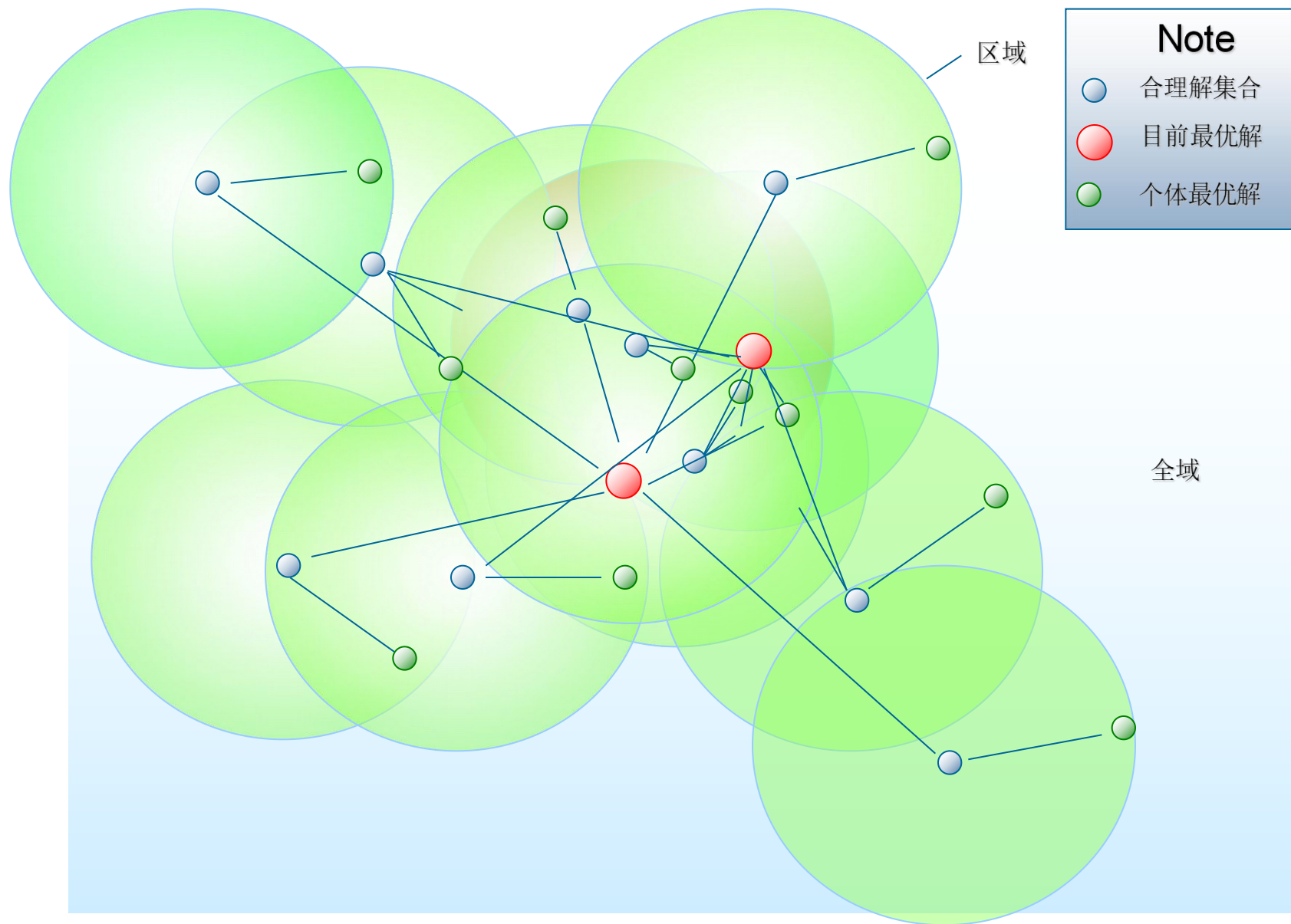
- $c_1 > 0, c_2 = 0$: 每个微粒执行局部搜索;
- $c_1 = 0, c_2 > 0$: 微粒群转化为一个随机爬山法;
- $c_1 = c_2 > 0$: 微粒逐渐移向 $\vec{p}_g$ 和 $\vec{p}_i$ 的加权均值;
- $c_2 > c_1$: 算法比较适合于单峰优化问题;
- $c_1 > c_2$: 算法比较适合于多峰优化问题。

PSO算法

PSO中粒子的位置更新



2维例子



压缩因子

Clerc和Kennedy [1] 建议把压缩因子引入微粒速度更新公式:

$$v_{ij}(t+1) = \chi[v_{ij}(t) + c_1 r_1(p_{ij}(t) - x_{ij}(t)) + c_2 r_2(p_{gj}(t) - x_{ij}(t))]$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

压缩因子

$$\text{其中, } \chi = \frac{2}{(2 - \varphi - \sqrt{\varphi^2 - 4\varphi})}, \quad \varphi = c_1 + c_2, \quad \varphi > 4.$$

通常, $c_1 = c_2 = 2.05$, 压缩因子 $\chi = 0.7298$ 。

[1] M. Clerc and J. Kennedy, "The particle swarm—explosion, stability, and convergence in a multidimensional complex space," IEEE Transactions on Evolutionary Computation, 2002, vol. 6, no. 1, pp. 58-73.

PSO的拓扑结构

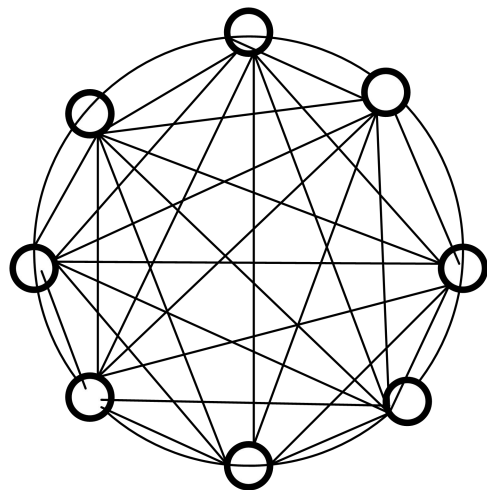
有两类PSO:

- ◆ **全局PSO**: 群体中每个微粒跟踪的两个极值为自身最佳位置 *pbest* 与群体最佳位置 *gbest*。
- ◆ **局部PSO**: 每个微粒跟踪自身最佳位置 *pbest* , 不跟踪群体的最佳位置, 而是跟踪 **其拓扑邻域中所有 K 个微粒的最佳位置 *lbest***, 其速度更新如下:

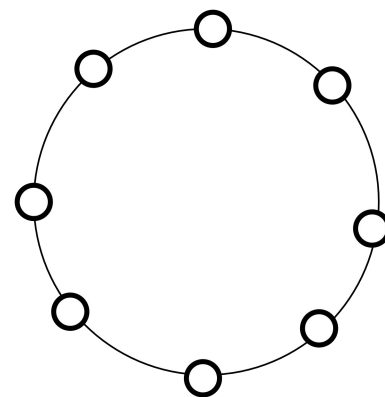
$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(p_{ij}(t) - x_{ij}(t)) + c_2r_2(\textcolor{red}{p}_{lj}(t) - x_{ij}(t))$$

$\vec{p}_l(t) = (p_{l1}, p_{l2}, \dots, p_{ld})$ 为局部邻域最佳位置。

拓扑结构:

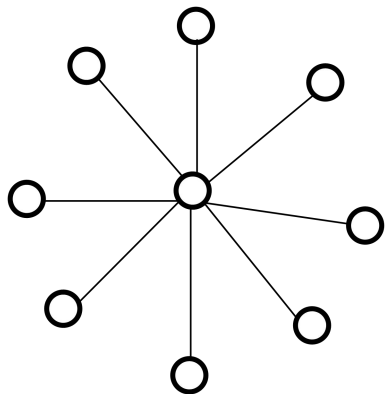


全连接拓扑, 即受 g_{best} 影响

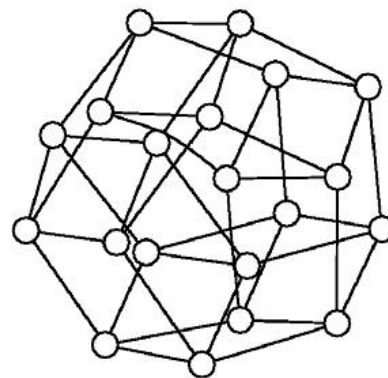


环形拓扑, $K=2$,
仅受与其紧邻的两个粒子的影响

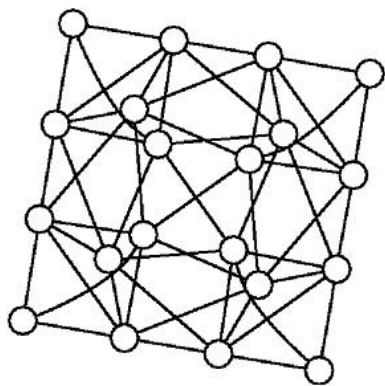
拓扑结构:



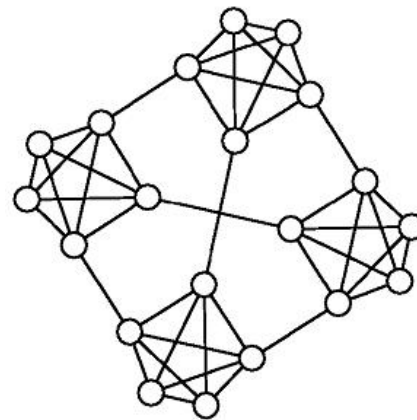
星状拓扑(star)



Von Neumann 拓扑



锥形拓扑(pyramid)



四聚类拓扑(clusters)

- ◆ 全连接拓扑信息传输速度快，相应地，PSO算法收敛速度快，但是易于早熟收敛。
- ◆ 环形拓扑信息传输速度最慢，相应地，PSO算法收敛速度慢，但是微粒有更多的机会发现最优解。
- ◆ Mendes和Kennedy (2002)在对比不同拓扑结构时发现：Von Neumann拓扑优于其它拓扑。
- ◆ Suganthan (1999)在PSO算法初期采用局部PSO, 后期采用全局版PSO;
- ◆ Hu和Eberhart (2002)提出了动态拓扑结构的概念。每次迭代时选择与当前微粒最近的 m 个微粒作为它的新邻域。

PSO的收敛性

- ◆ Van den Bergh(2002)、Trelea(2003)以及Bergh和Engelbrecht (2006)已经证明PSO能够收敛到一个平衡状态。
- ◆ 对于全局版PSO 算法，如果

$$0 < w < 1, c_1, c_2 > 0, 0 < c_1 + c_2 < 4(1 + w)$$

那么，

$$\lim_{t \rightarrow \infty} \vec{x}_i(t) = \frac{c_1 \vec{p}_i(t) + c_2 \vec{p}_g(t)}{c_1 + c_2}$$

- ◆ 这意味着随着迭代次数的增加所有微粒将收敛到一个点。
- ◆ 但是，这不意味着微粒个体最优点和全局最优点的加权和就是问题的最优解。

PSO的收敛性

- ◆ 事实上，存在微粒群收敛到局部稳定点的可能。该局部稳定点不是问题的全局最优点，甚至不是问题的局部最优点。

进一步解释：

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(p_{ij}(t) - x_{ij}(t)) + c_2r_2(p_{gj}(t) - x_{ij}(t))$$

$$x_{ij}(t+1) = x_{ij}(t) + v_{ij}(t+1)$$

如果式中 $\vec{p}_i(t) = \vec{p}_g(t) = \vec{x}_i(t)$ ，那么，微粒仅依靠 $w\vec{v}_i(t)$ 更新其位置。

进一步，如果上述状态持续足够代数后，那么

$$w\vec{v}_i(t) \rightarrow 0 \quad (t \rightarrow \infty), \quad \text{where} \quad 0 \leq w < 1$$

因此，有 $\vec{x}_i(t) \rightarrow a \quad (t \rightarrow \infty)$

PSO的收敛性

- ◆ 防止所有或者部分微粒长期接近某微粒的全局最优点（非所有粒子的全局最优点），即

$$\sum_{i=1}^N |\vec{p}_g(t) - \vec{x}_i(t)| > c, \quad c > 0$$

部分典型方法：

Dissipative PSO (*Xie, et al., 2002*)----增加微粒群的多样性;

PSO with mutation (*Stacey, et al., 2004*)----重新初始化已经收敛的微粒;

PSO with passive congregation (*He et al., 2004*)----提高微粒群的多样性;

Speciation-based PSO (*Li et al., 2006*)----根据微粒之间相似程度，把整个微粒群划分为若干类。

Self-organizing PSO (*Ratnaweera, et al. 2004*) ----改进算法的控制参数;

Multi-swarm parallel PSO, MPPSO (*Belal M et al., 2004*)-----利用多种群思想。

PSO的收敛性

- ◆ 利用具有较强局部搜索能力的算法进一步细化/开发PSO所得结果。

一些方法：

差分算法(Differential Evolution, DE) (*Zhang and Xie, 2003*)

遗传算法 (Genetic algorithm, GA) (*Matthew et al., 2005*);

爬山法;

模拟退火(Simulated annealing, SA) (*Nasser Sadati et al., 2006*);

单纯形法 (Simplex method, SM) (Fan S K et al., 2007).

二进制PSO

- ◆ 最初的PSO算法是从处理连续优化问题中发展起来的。
- ◆ Kennedy和Eberhart[1]首次将实数PSO扩展为二进制PSO。
- ◆ 微粒位置为二进制向量，微粒速度仍为浮点向量；
- ◆ 微粒速度被逻辑函数 $s(v)$ 转化为判断位置项选择0还是1的概率。

[1] J. Kennedy, R. C. Eberhart. "A discrete binary version of the particle swarm algorithm," Proceedings of International Conference on System, Man, and Cybernetics, 1997, Orlando, FL, pp. 4104-4109.

二进制PSO

更新公式如下：

$$v_{ij}(t+1) = wv_{ij}(t) + c_1r_1(p_{ij}(t) - x_{ij}(t)) + c_2r_2(p_{gj}(t) - x_{ij}(t))$$

$$x_{ij}(t+1) = \begin{cases} 0, & \text{if } s(v_{ij}(t+1)) < U(0,1), \\ 1, & \text{otherwise.} \end{cases}$$

其中

$$s(v_{ij}) = \frac{1}{1 + e^{-v_{ij}}}$$

为了防止 $s(v)$ 饱和，Kennedy等建议将 v_{ij} 限制在 $[-4,4]$ 之间。

PSO与遗传算法的区别

- ◆ 遗传算法强调“**适者生存**”，不好的个体在竞争中被淘汰；PSO强调“**协同合作**”，不好的个体通过学习向好的方向转变。
- ◆ 遗传算法中最好的个体通过产生更多的后代来**传播基因**；PSO中的最好个体通过吸引其它个体向它靠近来**施加影响**。
- ◆ 遗传算法的选择概率只与上一代群体相关，而与历史无关，群体的信息变化过程是一个Markov链过程；而PSO中的个体除了有位置 and 速度外，还有着过去的历史信息（*pBest*、*gBest*）。

PSO的优点和缺点

优点:

- ◆ 易于实现;
- ◆ 可调参数较少;
- ◆ 所需种群或微粒群规模较小;
- ◆ 计算效率高, 收敛速度快。

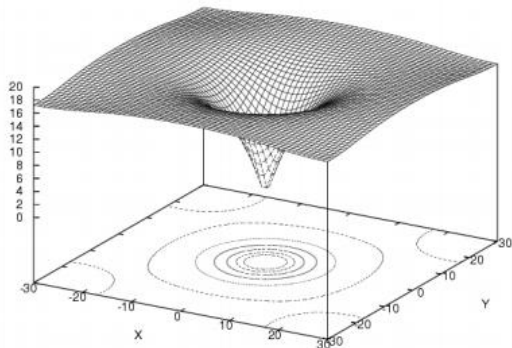
缺点:

- ◆ 基本PSO虽然收敛速度快, 但有时会陷入局部最优。

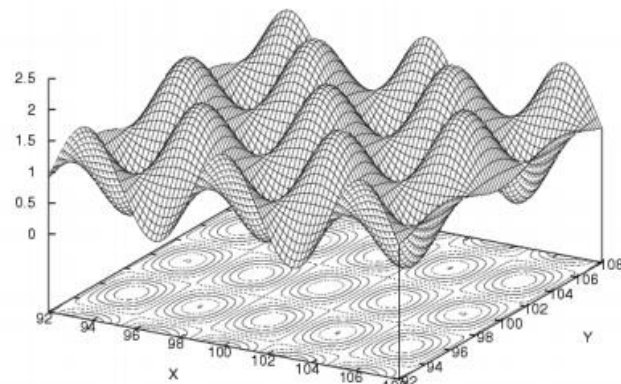
PSO改进研究

- ◆ 位置和速度更新公式
- ◆ 多种群PSO
- ◆ 种群拓扑结构
- ◆ 与其他智能优化算法的混合
- ◆ 拓展PSO算法用于：
 - 多目标优化
 - 约束优化
 - 离散优化
 - 动态优化

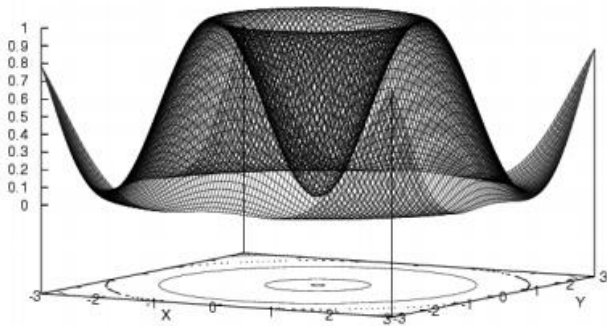
所用的测试函数（几个例子）



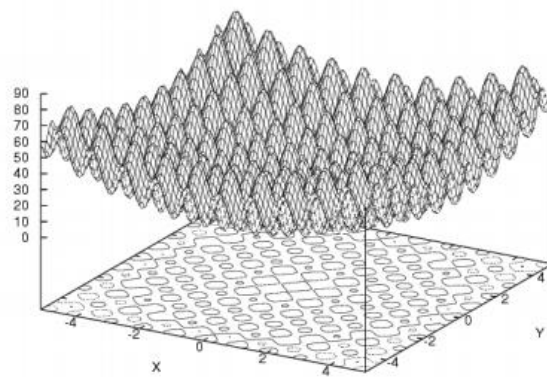
Ackley



Griewank



Schaffer



Rastrigin

PSO应用研究

- ◆ 函数优化——数值函数优化，动态、多峰值、多目标优化
- ◆ 组合优化——旅行商问题、车辆路由问题、布局优化问题等
- ◆ 生产调度——工件加工问题
- ◆ 自动控制——智能控制器优化、最优控制器的设计
- ◆ 机器人学——路径规划、协调控制
- ◆ 机器学习——分类、聚类、数据挖掘
- ◆ 图像处理——图像识别、检测
- ◆ 机械设计——形状、结构、参数的设计
- ◆ 神经网络训练
- ◆ 系统参数辨识等